1 Introductry Digital Concepts

Electronic circuits can be divided into two broad categories, digital and analog. Digital electronics involves quantities with discrete values, and analog electronics involves quantities with continuous values.

An analog quantities is one having continuous values. A digital quantity is one having a discrete set of values. Most things that can be measured quantitatively appear in nature in analog form. For example, the air temperature changes over a continuous range of values. During a given day, the temperature does not go from, say, 70° to 71 ° instantaneously; it takes on all the infinite values in between. If you graphed the temperature on a typical summer day, you would have a smooth, continuous curve similar to the curve in Fig.(1-1). Other examples of analog quantities are time, pressure, distance, and sound.

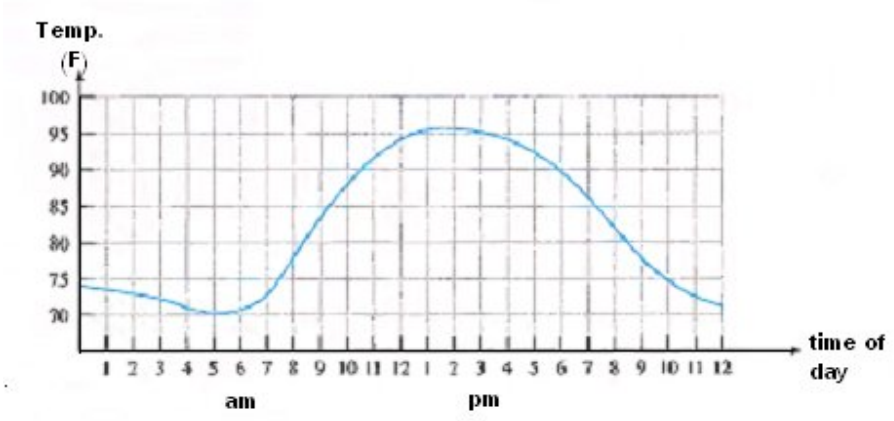


Fig.(1-1) Graph of an analog quantity.

Suppose, you just take a temperature reading every hour. Now, you have sampled values representing the temperature at discrete points in time (every hour) over a 24-hour period, as indicated in Fig.(1-2). You have effectively converted an analog quantity to a form that can now be digitized by representing each sample value by a digital code.

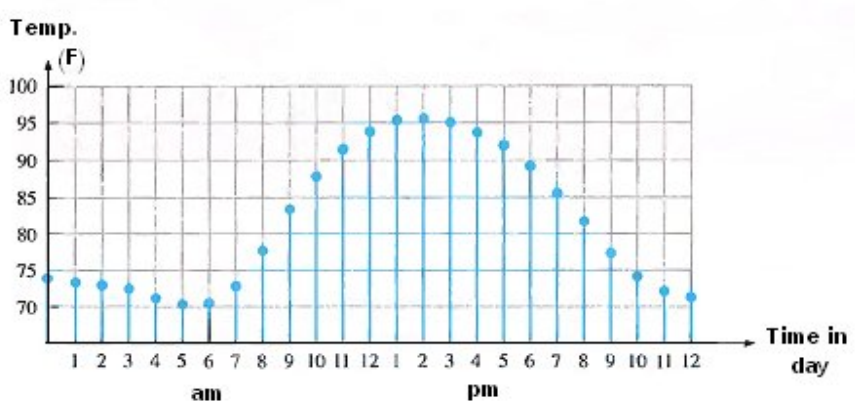


Fig.(1-2) Sampled-value representation of the analog quantity in Fig.(1-1).

The Digital Advantages

- Digital data can be processed and transmitted more efficiently and reliably than analog data.
- Digital data has a great advantage when storage is necessary. For example, music when converted to digital form can be stored more compactly and reproduced with greater accuracy and clarity than is possible when it is an analog form.
- Noise (unwanted voltage fluctuations) does not affect digital data nearly as much as it does analog signals.
- Digital systems are used in communication, business transaction, traffic control, space guidance, medical treatment, weather monitoring, the

internet, and many other commercial, industrial, and scientific enterprises.

We have digital telephones, digital TV's, digital versatile discs, digital cameras……..

An Analog Electronic System

A pubic address system, used to amplify sound so that it can be heard by a large audience.

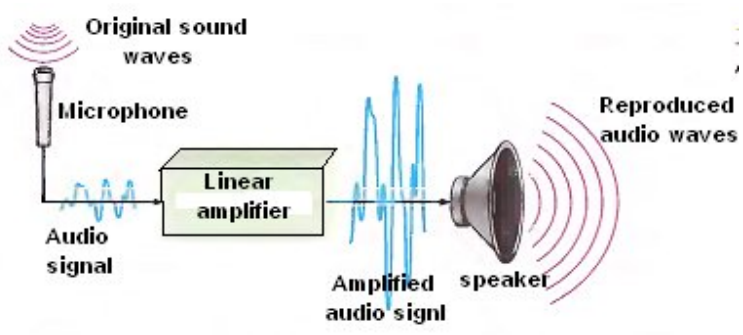


Fig.(1-3) A basic audio public address system.

Digital / Analog Electronic System

The compact disc (CD) player is an example of a system in which both digital and analog circuits are used. Music in digital form is stored on the compact disc. A laser diode optical system picks up the digital data from the rotating disc and transfers it to the (DAC) digital-to-analog converter. The output analog signal is amplified and sent to the speaker.

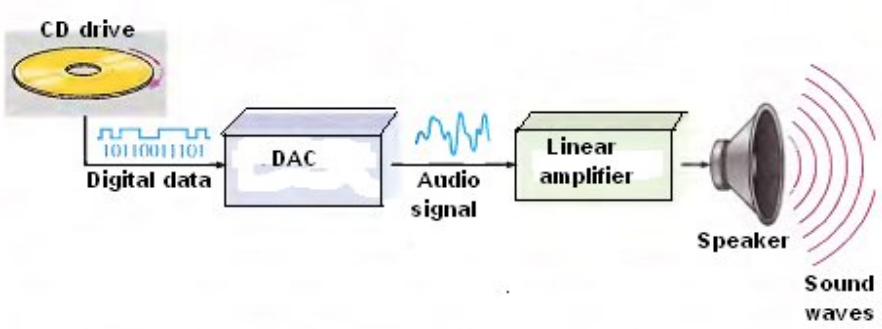


Fig.(1-4) Basic principle of a CD player.

Binary Digits

The two digits in the binary system, 1 & 0 are called bits, which is a contraction of the words binary digit.

In digital circuits, two different voltage levels are used to represent the two bits. A 1 is represented by the higher voltage (HIGH) and a 0 is represented by the lower voltage level (LOW). This is called positive logic.

HIGH = 1 : LOW = 0

Codes: groups of bits (combinations of 1s and 0s). Codes are used to represent numbers, letters, symbols, instructions and anything else required in a given application.

Logic Levels

The voltages used to represent a 1 and a 0 are called logic levels. Ideally, one voltage level represents a 1 and another voltage level represents a 0. In practical digital circuit, a HIGH can be any voltage between a specified minimum value and a specified maximum value. Likewise, a LOW can be any voltage between a specified minimum and a specified maximum. There can be no overlap between the accepted HIGH levels and the accepted LOW levels.

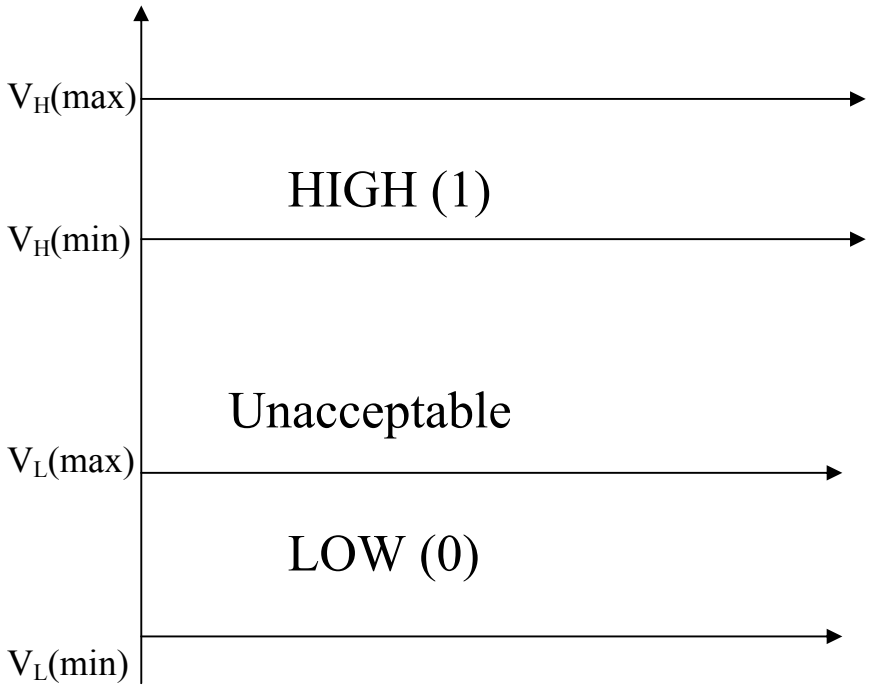


Fig.(1-5) Logic level ranges of voltage for a digital circuit.

Digital Waveforms

1. The positive-going pulse: is generated when the voltage (or current) goes from its normally LOW level to its HIGH level and then back to its LOW level.
2. The negative-going pulse is generated when the voltage goes from its normally HIGH level to its LOW level and back to its HIGH level.

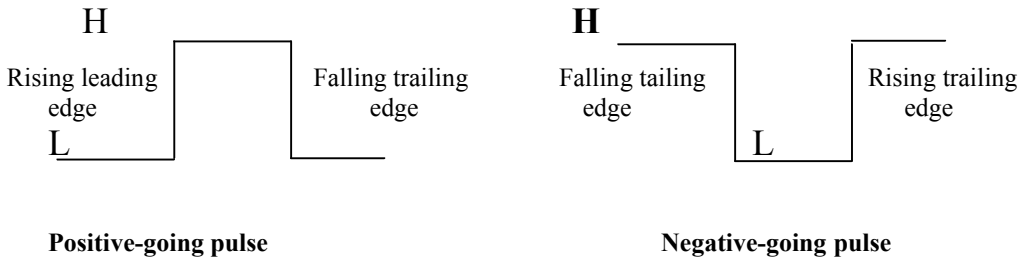


Fig.(1-6) Ideal pulses

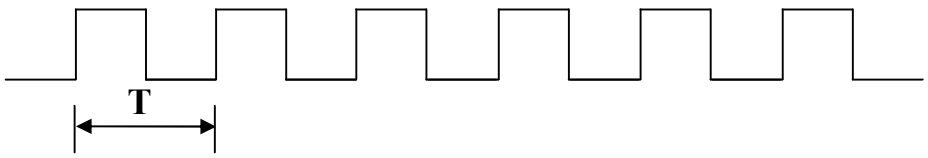
Q *What about nonideal pulse?*

Q *What is your information about rise time, fall time and pulse width?*

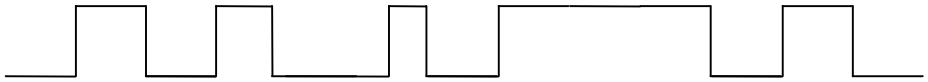
Pulse trains: series of pulses.

Periodic pulse waveform: is one that repeats itself at a fixed interval, called a period (T). Frequency (F) = 1/T , measured in (Hz), when (T) in (sec).

Nonperiodic pulse waveform does not repeat itself at fixed intervals and may be composed of pulses of randomly differing pulse width and/or randomly differing time intervals between the pulses.



(a)



(b)

Fig.(1-7) Pulse waveforms.

An important characteristic of a periodic digital waveform is its duty cycle. The duty cycle is defined as the ratio of the pulse width (t_w) to the period (T):

$$\text{Duty cycle} = (t_w/T) \times 100\%$$

Timing Diagrams

A timing Diagram is a graph of digital waveforms showing the actual time relationship of two or more waveforms and how each waveforms and how each waveform changes in relation to the others.

The clock: In digital systems, all waveforms are synchronized with a base timing waveform called the clock. It is a periodic waveform in which each interval between pulses (the period) equals the time for one bit. The clock waveform itself does not carry information.

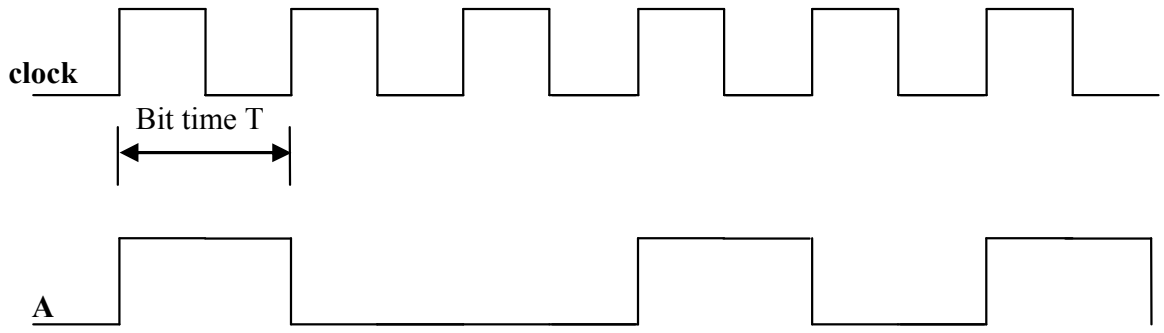


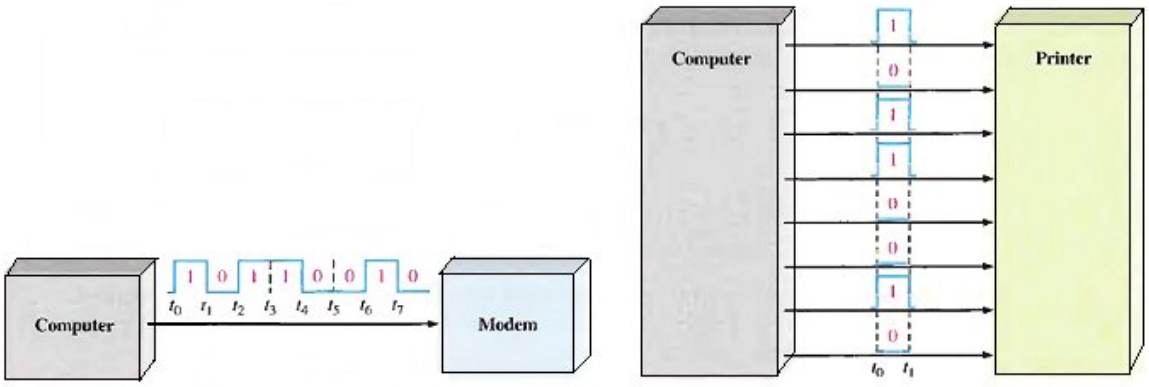
Fig.(1-8) A simple timing diagram.

Data Transfer

1. Serial.
2. Parallel.

When bits are transferred in serial from one point to another, they are sent one bit at a time along a single conductor. To transfer eight bits in series, it takes eight time intervals.

When bits are transferred in parallel form, all the bits in a group are sent out on separate lines at the same time. There is one line for each bit.



(a) Serial transfer. **(b) Parallel transfer**

Fig.(1-9) Illustration of serial and parallel transfer of binary data. Only the data lines are shown.

Basic Logic Operations

Gorge Boole (1850s)

Three basic logic operations are indicated by standard distinctive symbols in Fig.(1-6) below:

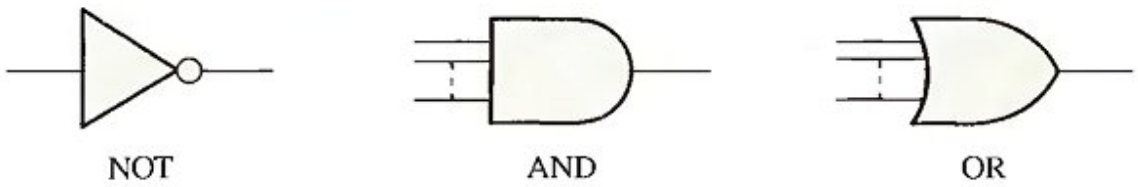


Fig.(1-10) Basic logic operations and symbols.

Logic gate is a circuit that performs a specified logic operation (AND, OR).

HIGH (true) : LOW (false)

NOT changes one logic level to the opposite level.

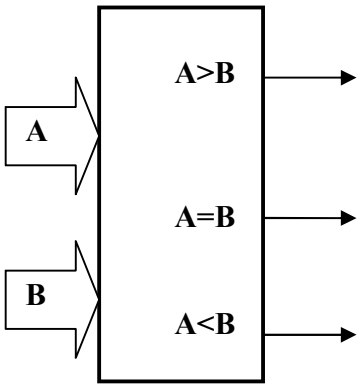
AND produces a HIGH output only if all the inputs are HIGH.

OR produces a HIGH output when any of the inputs is HIGH.

Basic Logic Functions

The three basic elements AND, OR, and NOT are combined to form more complex logic circuits that perform many useful operations and that are used to build complete digital systems.

1. The arithmetic functions:
    - Addition
    - Subtraction
    - Multiplication
    - Division
2. The comparison function:



3. The code conversion function: A code is a set of bits arranged in a unique pattern and used to represent specified information.

    Binary ⟶ BCD

    Binary ⟶ Gray code

4. The encoding function (Encoder): converts information e.g. decimal number into some coded form, like binary code.

5. The decoding function (Decoder): converts coded information into a noncoded form.

6. The data selection function:

   Multiplexer (MUX): is a logic circuit that switches digital data from several input lines onto a single output line in a specified time sequence.

   Demultiplexer (DEMUX): is a logic circuit that switches digital data from one input line to several output lines in a specified time sequence.

7. The storage function

   Flip-Flop, Registers, Semiconductor memories, Magnetic disks, ets…

8. The counting function.

Digital Integrated Circuits

A monolithic integrated circuit (IC) is an electronic circuit that is constructed entirely on a single small chip of silicon.

IC packages are classified according to the way they are mounted on printed circuit (PC) boards:

- The through-hole type packages have pins (leads) that are inserted through holes in the PC board and can be soldered to conductors on the opposite side. (DIP) dual-in-line package.
- The surface-mount technology (SMT) newer, space-saving, holes are unnecessary. (SOIC) small-outline integrated circuit.

ICs are classified according to their complexity:

1- Small-scale integration (SSI) < 12 gate/chip. They include basic gates and flip-flops.

2- Medium-scale integration (MSI) 12 – 99 gate/chip. They include encoders, decoders, counters, registers, multiplexers, small memories.
3- Large-scale integration (LSI) 100 – 9999 gate/chip, including memories.
4- Very large-scale integration (VLSI) 10,000 – 99,999 gate/chip, including (memories and microprocessors).
5- Ultra large-scale integration (ULSI) >100,000. It describes very large memories, larger microprocessors, and large single-chip computers.

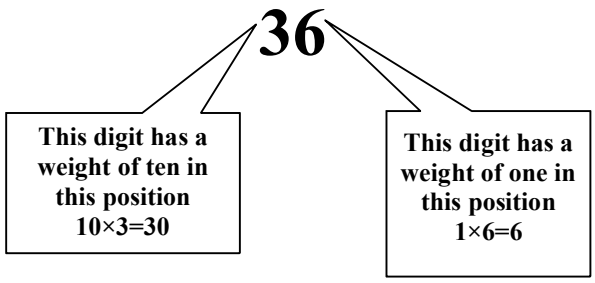
ICs are classified according to their operation into Analog IC & Digital IC or both in one.

ICs are classified according to the type of transistors (BJT) bipolar junction transistor and (MOSFET) metal-oxide semiconductor field- effect transistors.

2 Number Systems

1- Decimal Numbers

In the decimal number system each of the ten digits 0 through 9, represents a certain quantity. These ten symbols (digits) don't limit you to expressing only ten different quantities, because you use the various digits in appropriate positions within a number to indicate the magnitude of the quantity.



The position of each digit in a decimal number indicates the magnitude of the quantity represented, and can be assigned a weight. The decimal number system is said to be of base, or radix, 10 because it uses 10 digits and the coefficients are multiplied by powers of 10. In general, a number with a decimal point is represented by a series of coefficients:

$$\mathbf{a_4\ a_3\ a_2\ a_1\ a_0\ .\ a_{-1}\ a_{-2}\ a_{-3}}$$

The coefficients a_j are any of the 10 digits (0, 1, 2, ……,9), and the subscript value j gives the place value and, hence, the power of 10 by which the coefficient must be multiplied.

$$10^4\times\mathbf{a_4}+10^3\times\mathbf{a_3}+10^2\times\mathbf{a_2}+10^1\times\mathbf{a_1}+10^0\times\mathbf{a_0}.\ 10^{-1}\times\mathbf{a_{-1}}+10^{-2}\times\mathbf{a_{-2}}+10^{-3}\times\mathbf{a_{-3}}$$

2- Binary Numbers

This is another way to represent quantities. The binary system is less complicated than the decimal system because it has only two digits. It's a base-2 system.

A binary digit, called a bit, has two values 0 & 1. Each coefficient $\mathbf{a_j}$ **is multiplied by** $\mathbf{2^j}$**,** and the results are adde**d** to obtain the decimal equivalent of the number. For example,

11010.11 is equal to 26.75 as follows:

$$1\times\mathbf{2^4} + 1\times\mathbf{2^3} + 0\times\mathbf{2^2} + 1\times \mathbf{2^1} + 0\times\mathbf{2^0} + 1\times \mathbf{2^{-1}} + 1\times \mathbf{2^{-2}}$$

$$\mathbf{16 + 8 + 0 + 2 + 0 + 0.5 + 0.25 = (26.75)_{10}}$$

In general, a number expressed in a base-r system has coefficients multiplied by powers of r.

$$r^n\times\mathbf{a_n} + r^{n-1}\times\mathbf{a_{n-1}} + \ldots.. + r^2\times\mathbf{a_2} + r^1\times\mathbf{a_1} + \mathbf{1}\times\mathbf{a_0} + \mathbf{r^{-1}}\times \mathbf{a_{-1}} + r^{-2}\times\mathbf{a_{-2}} + \mathbf{\ldots..} + r^{-m}\times\mathbf{a_{-m}}$$

Now, let us begin to count in the binary system:

One bit	2 values	0, 1
Two bits	4 values	00, 01, 10, 11
Three bits	8 values	000, 001, 010, 011, 100, 101, 110, 111.
Four bits	16 values	0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111.

And so on……..

Decimal number	Binary number 8421
0	0000 ◄—— (LSB) least significant bit
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
	↑
	(MSB) most significant bit

LSB (right-most bit) has a weight of $2^0 = 1$.

MSB (left- most bit) has a weight of $2^3 = 8$.

In general, with n – bit, you can count up to a number equal to:

Largest decimal number = $2^n - 1$

When n = 5, you can count from 0 – 31, (32 values). Max. number = $(31)_{10}$.

The weight structure of a fractional binary number is

2^{n-1}…..2^3 2^2 2^1 2^0 . 2^{-1} 2^{-2} 2^{-3} …… 2^{-n}

…... 8 4 2 1 . 0.5 0.25 0.125 ……..

3 – Hexadecimal Numbers

It has (16) digits and is used primarily as a compact way of displaying or writing binary numbers, it's very easy to convert between binary and hexadecimal numbers.

Decimal number	Binary number 8421	Hexadecimal number
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

How do you count in hexadecimal once you get to F? Simply start over with another column and continue as follows:

10 11 12 13 14 15 16 17 18 19

1A 1B 1C 1D 1E 1F 20 21 22 23

24 25 26 27 28 29 2A 2B 2C 2D…….

With two hexadecimal digits, you can count up to FF which in decimal 255.

$(100)_{16}$ = $(256)_{10}$

$(101)_{16}$ = $(257)_{10}$

$(FFF)_{16}$ = $(4095)_{10}$

$(FFFF)_{16}$ = $(65535)_{10}$

4- Octal Numbers

This system provides a convenient way to express binary numbers and codes (like Hex. Number system), it is used less frequently than hexadecimal.

The octal number system is composed of eight digits, which are:

0 1 2 3 4 5 6 7

To count above 7, begin another column and start over

10 11 12 13 14 15 16 17

20 21 22 23 24 25 26 27

30 31 32 33 34 35 36 37……

$(15)_8$ = $(13)_{10}$ = $(D)_{16}$

5- Binary Coded Decimal (BCD)

The 8421 code is a type of binary coded decimal. It means that each decimal digit 0 through 9 is represented by a binary code of four bits:

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

{1010, 1011, 1100, 1101, 1110, 1111} are invalid in 8421 BCD code.

10 = 0001 0000 : 32 = 0011 0010

6- The Gray Code

- is unweigthed.
- is not an arithmetic code.
- it exhibits only a single bit change from one code number to the next.

Decimal number	Binary number	Gray code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

7- ASCII

(The American Standard Code for Information Interchange) pronounced "askee".

- It has 128 characters and symbols represented by a 7-bit binary code.
- 32 ASCII characters are nongraphic never printed or displayed and used only for control purposes.
- Others are graphic symbols for printing and displaying.

8- Exess-3 Code

Is unweighted code in which each code is obtained from the corresponding binary value plus 3.

Binary – to – Decimal Conversion

The decimal value of any binary number can be found by adding the weights of all bits that are 1 and discarding the weights of all bits that are 0.

Example

Convert the binary number 1101101 to decimal.

Solution

$$2^6 \times 1 + 2^5 \times 1 + 2^4 \times 0 + 2^3 \times 1 + 2^2 \times 1 + 2^1 \times 0 + 2^0 \times 1$$

$$64 + 32 + 0 + 8 + 4 + 0 + 0 + 1 = (109)_{10}$$

Example

Convert 0.1011 to decimal.

Solution

$$2^0 \times 0 + 2^{-1} \times 1 + 2^{-2} \times 0 + 2^{-3} \times 1 + 2^{-4} \times 1$$

$$0 + 0.5 + 0.125 + 0 + 0.0625 = (0.6875)_{10}$$

Decimal – to – Binary Conversion

- Sum – of – Weights Method: determine the set of binary weights whose sum is equal to the decimal number.

Example: Convert 9 to binary.

Solution : 9= 8 + 1 "choosing numbers with power of 2 "

Binary weights are: …..32 16 8 4 2 1

1 0 0 1

So $(9)_{10} = (1001)_2$

Example

Convert the following

12 = 8 + 4 = 1100

25 = 16 + 8 + 1 = 11001

82 = 64 + 16 + 2 = 1010010.

- Repeated Division -by- 2 Method:

12/2 = 6 reminder = 0 this is the LSB

6/2 = 3 reminder = 0

3/2 = 1 reminder = 1

1/2 = 0 reminder = 1 this is the MSB

Stop when the whole number quotient is 0.

Example: convert 45 to binary.

Solution:

45/2 = 22 reminder = 1

22/2 = 11 reminder = 0

11/2 = 5 reminder = 1

5/2 = 2 reminder = 1

2/2 = 1 reminder = 0

1/2 = 0 reminder = 1

$(45)_{10} = (101101)_2$

What about decimal numbers with fractions?

Binary weights . 0.5 0.25 0.125 0.0625

(0.625)10 = 0.5 + 0.125

= (0.101)2 (by using sum – of – weight method)

OR by using repeated MULTIPLECTION - by – 2 method

0.625 × 2 = 1.25	1	this is the MSB
0.25 × 2 = 0.5	0	
0.5 × 2 = 1.00	1	this is the LSB

So $(0.625)_{10}$ = $(0.101)_2$

MSB LSB

Binary to Hexadecimal Conversion

Simply break the binary number into 4-bit groups, starting at the right-most bit and replace each 4-bit group with the equivalent hexadecimal symbol.

Example

- 1100101001010111 = 1100 1010 0101 0111
  C A 5 7

- Note: each group must be four bits.

Hexadecimal – to – Binary Conversion

Reverse the process and replace each hexadecimal symbol with the appropriate four bits.

Example

- 10AF = 1 0 A F
  0001 0000 1010 1111

So $(10AF)_{16} = (0001000010101111)_2$

Hexadecimal to Decimal Conversion

Repeated division of a decimal number by 16 will produce the equivalent hexadecimal number, formed by the reminders of the division. The firs reminder produced is the LSD. Each successive division by 16 yields a reminder that becomes a digit in the equivalent hexadecimal number.

Note : when a quotient has a fractional part, the fractional part is multiplied by the divisor to get the remainder.

Example

$(650)_{10} = (\ ?\ )_{16}$

Solution

650/16 = 40 reminder = 10 = A this is the LSD

40/16 = 2 reminder = 8 = 8

2/16 = 0 reminder = 2 = 2 this is the MSD

So $(650)_{10} = (28A)_{16}$

Octal – to – Decimal Conversion

$$(2374)_8 = 2 \times 8^3 + 3 \times 8^2 + 7 \times 8^1 + 4 \times 8^0$$
$$= 1024 + 192 + 56 + 4$$
$$= (1276)_{10}$$

Decimal – to – Octal Conversion

$(359)_{10} = (\ )_8$

359/8 = 44 reminder = 7 this is the LSD

44/8 = 5 reminder = 4

5/8 = 0 reminder = 5 this is the MSD

So $(359)_{10} = (547)_8$

Octal – to – Binary Conversion

Replace each octal digit with the appropriate three bits.

Example

$(25)_8 = (010\ 101)_2$

$(7526)_8$ = 7 5 2 6

111 101 010 110

$(7526)_8 = (111101010110)_2$

Binary – to – Octal Conversion

Start with the right – most group of three bits and moving from right to left, convert each 3-bit group to the equivalent octal digit.

Example

11010000100 = 011 010 000 100

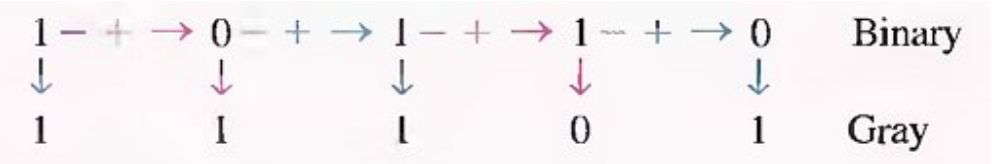
3 2 0 4

$(11010000100)_2 = (3204)_8$

Binary – to – Gray Code Conversion

- The most significant bit (left-most) in the Gray code is the same as the corresponding MSB in the binary number.
- Going from left to right, add each adjacent pair of binary code bits to get the next gray code bit.

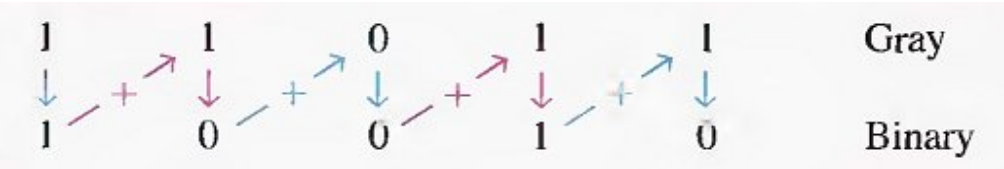
For example, the conversion of the binary number 10110 to Gray code is as follows:



Gray – to – Binary Conversion

- The most significant bit (left-most) in the binary code is the same as the corresponding bit in the Gray code.
- Add each binary code bit generated to the Gray code bit in the next adjacent position.

For example, the conversion of the Gray code word 11011 to binary is as follows:



Binary Arithmatic

1- Binary Addition

0 + 0 = 0 with a carry = 0

0 + 1 = 1 with a carry = 0

1 + 0 = 1 with a carry = 0

1 + 1 = 0 with a carry = 1

Example

11 + 11 = 110 because

11	3
+ 11	+ 3
110	6

110	6
+ 100	+ 4
1010	10

2- Binary Subtraction

0 – 0 = 0 with a barrow = 0

0 – 1 = 1 with a barrow = 1

1 – 0 = 1 with a barrow = 0

1 – 1 = 0 with a barrow = 0

Example

11	3
- 01	- 1
10	2

3- Binary Multiplication

$0 \times 0 = 0$

$0 \times 1 = 0$

$1 \times 0 = 0$

$1 \times 1 = 1$

It involves forming partial products, shifting each successive partial product left one place and then adding all the partial products.

Examples

```
   11          3
 ×11         ×3
 ----        ----
   11          9
+ 11
 ----
 1001
```

```
   111          7
 ×101         ×5
 ----         ----
   111         35
  000
+111
 ------
100011
```

4-Binary Division

This operation follows the same procedure as division in decimal number system.

Example

$110 \div 11 = 10$ $\quad\quad$ $6 \div 3 = 2$

```
      10
   ------
11 | 110
     -11
   ----
     000
```

```
        11
     -----
10 |  110
     - 10
     ----
       010
     -  10
     ----
       000
```

1’s and 2’s Complement of Binary Numbers

The l's complement and the 2's complement of a binary number are important because they permit the representation of negative numbers. The method of 2's complement arithmetic is commonly used in computers to handle negative numbers.

Finding the 1's Complement

The l's complement of a binary number is found by changing all 1s to 0s and all 0s to 1s, as illustrated below:

10110010 Binary number

↓↓↓↓↓↓↓↓

01001101 1 's complement

The simplest way to obtain the l's complement of a binary number with a digital circuit is to use parallel inverters (NOT circuits), as shown in Fig.(2-1) for an 8-bit binary number.

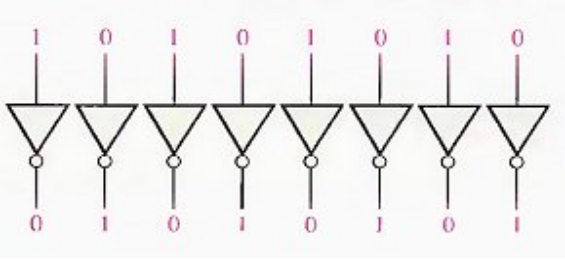


Fig.(2-1) Example of inverters used to obtain the 1 's complement of a binary number.

Finding the 2' s Complement

The 2's complement of a binary number is found by adding 1 to the LSB of the l's complement.

2's complement = (l's complement) + 1

Example

Find the 2's complement of 10110010.

Solution

10110010	Binary number
01001101	l's complement
+ 1	add 1
01001110	2's complement

An alternative method of finding the 2's complement of a binary number is as follows:

1. Start at the right with the LSB and write the bits as they are up to and including the first 1.
2. Take the 1's complements of the remaining bits.

Example:

Find the 2's complement of 10111000 using the alternative method.

Solution

10111000	Binary number
01001000	2's complement

The 2's complement of a negative binary number can be realized using inverters and an adder, as indicated in Fig.(2-2). This illustrates how an 8-bit number can be converted to its 2's complement by first inverting each bit (taking the l's complement) and then adding 1 to the l's complement with an adder circuit.

To convert from a l's or 2's complement back to the true (uncomplemented) binary form, use the same two procedures described previously. To go from the l's complement back to true binary, reverse all the bits. To go from the 2's complement form back to true binary, take the 1's complement of the 2's complement number and add 1 to the least significant bit.

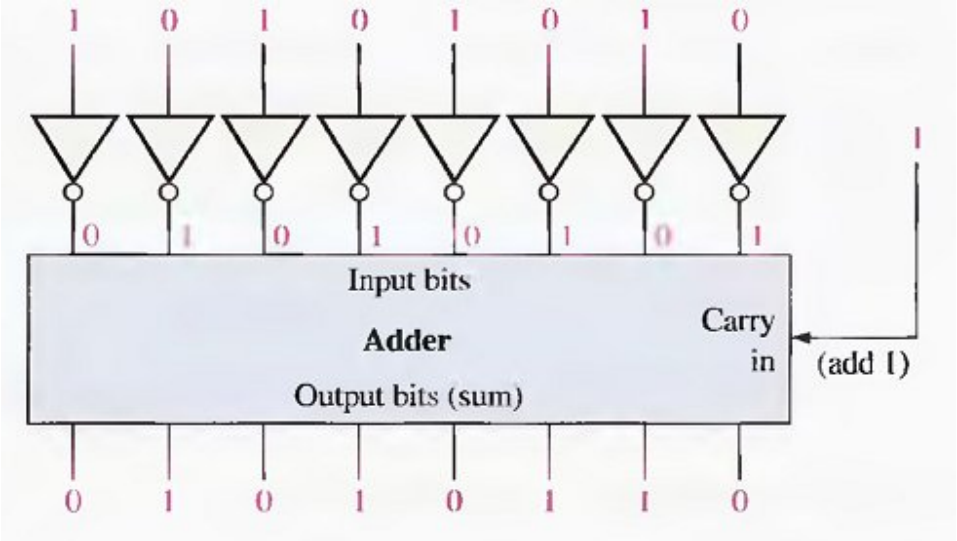


Fig.(2-2) Example of obtaining the 2's complement of a negative binary number.

Signed Numbers

Digital systems, such as the computer, must be able to handle both positive and negative numbers. A signed binary number consists of both sign and magnitude information. The sign indicates whether a number is positive or negative, and the magnitude is the value of the number. There are three forms in which signed integer (whole) numbers can be represented in binary: sign-magnitude, l's complement, and 2' complement. Of these, the 2's complement is the most important and the sign-magnitude is the least used.

The Sign Bit

The left-most bit in a signed binary number is the sign bit, which tells you whether the number is positive or negative. A 0 sign bit indicates a positive number, and a 1 sign bit indicates a negative number.

A 0 sign bit indicates a positive number, and a 1 sign bit indicates a negative number.

Sign-Magnitude Form

When a signed binary number is represented in sign-magnitude, the left-most bit is the sign bit and the remaining bits are the magnitude bits. The magnitude bits are in true (uncomplemented) binary for both positive and negative numbers. For example, the decimal number + 25 is expressed as an 8-bit signed binary number using the sign-magnitude form as 00011001.

The decimal number - 25 is expressed as 10011001. Notice that the only difference between + 25 and - 25 is the sign bit because the magnitude bits are in true binary for both positive and negative numbers.

In the sign-magnitude form, a negative number has the same magnitude bits as the corresponding positive number but the sign bit is a 1 rather than a zero.

l' s Complement Form

Positive numbers in 1's complement form are represented the same way as the positive sign-magnitude numbers. Negative numbers, however, are the l's complements of the corresponding positive numbers. For example. using eight bits. the decimal number -25 is expressed as the 1' s complement of + 25 (0001100 I) as 11100110.

In the l's complement form, a negative number is the l's complement of the corresponding positive number.

2 ' s Complement Form

Positive numbers in 2's complement form are represented the same way as in the sign-magnitude and l's complement forms. Negative numbers are the 2's complements of the corresponding positive numbers. Again, using eight bits, let's take decimal number -25 and express it as the 2's complement of +25 (00011001).

11100111

In the 2's complement form, a negative number is the 2's complement of the corresponding positive number.

Example:

Express the decimal number - 39 as an 8-bit number in the sign-magnitude, 1's complement, and 2's complement forms.

Solution

First, write the 8-bit number for + 39.

00100111

In the sign-magnitude form, - 39 is produced by changing the sign bit to a 1 and leaving the magnitude bits as they are. The number is

10100111

In the 1's complement form, -39 is produced by taking the l's complement of +39 (00100111).

11011000

In the 2's complement form, - 39 is produced by taking the 2's complement of +39 (00100111) as follows:

```
  11011000        1's complement
+        1
-----------
  11011001        2's complement
```

The Decimal Value of Signed Numbers

Sign-magnitude: Decimal values of positive and negative numbers in the sign-magnitude form are determined by summing the weights in all the magnitude bit positions where there are 1s and ignoring those positions where there are zeros. The sign is determined by examination of the sign bit.

Example:

Determine the decimal value of this signed binary number expressed in sign-magnitude: 10010101

Solution

The seven magnitude bits and their powers-of-two weights are as follows:

2^6 2^5 2^4 2^3 2^2 2^1 2^0

0 0 1 0 1 0 1

Summing the weights where there are 1s,

16 + 4 + 1 = 21

The sign bit is 1; therefore, the decimal number is - 21.

1‘s Complement: Decimal values of positive numbers in the l's complement form are determined by summing the weights in all bit positions where there are 1s and ignoring those positions where there are zeros. Decimal values of negative numbers are determined by assigning a negative value to the weight of the sign bit, summing all the weights where there are 1s, and adding 1 to the result.

Example:

Determine the decimal values of the signed binary numbers expressed in l's complement: (a) 00 010 111 (b) 11 101 000

Solution:

(a) The bits and their powers-of-two weights for the positive number are as follows:

-2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

0 0 0 1 0 1 1 1

Summing the weights where there are 1s, $16 + 4 + 2 + 1 = +23$

(b) The bits and their powers-of-two weights for the negative number are as follows. -2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

1 1 1 0 1 0 0 0

Notice that the negative sign bit has a weight of -2^7 or -128.

Summing the weights where there are 1s, $-128 + 64 + 32 + 8 = -24$

Adding 1 to the result, the final decimal number is $-24 + 1 = -23$.

2's Complement: Decimal values of positive and negative numbers in the 2's complement form are determined by summing the weights in all bit positions where there are 1s and ignoring those positions where there are zeros. The weight of the sign bit in a negative number is given a negative value.

Example:

Determine the decimal values of the signed binary numbers expressed in 2's complement:

(a) 01010110 (b) 10101010

Solution:

(a) The bits and their powers-of-two weights for the positive number are as follows:

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	1	0	1	1	0

Summing the weights where there are 1s,

$64 + 16 + 4 + 2 = +86$

(b) The bits and their powers-of-two weights for the negative number are as follows. Notice that the negative sign bit has a weight of $-2^7 = -128$.

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	1	0	1	0	1	0

Summing the weights where there are 1s, $-128 + 32 + 8 + 2 = -86$

Range of Signed Integer Numbers That Can Be Represented

We have used 8-bit numbers for illustration because the 8-bit grouping is common in most computers and has been given the special name byte. With one byte or eight bits, you can represent 256 different numbers. With two bytes or sixteen bits, you can represent 65,536 different numbers. With four bytes or 32 bits, you can represent 4.295×10^9 different numbers. The formula for finding the number of different combinations of n bits is

$$\text{Total combinations} = 2^n$$

For 2's complement signed numbers, the range of values for n-bit numbers is

$$\text{Range} = -(2^{n-1}) \text{ to } +(2^{n-1} - 1)$$

where in each case there is one sign bit and (n – 1) magnitude bits. For example, with four bits you can represent numbers in 2's complement ranging from $-(2^3) = -8$ to $2^3 - 1 = +7$. Similarly, with eight bits you can go from -128 to + 127, with sixteen bits you can go from - 32,768 to + 32,767, and so on.

3 Logic Gates

THE INVERTER

The inverter (NOT circuit) performs the operation called inversion or complementation. The inverter changes one logic level to the opposite level. In terms of bits, it changes a 1 to a 0 and a 0 to a 1.

Standard logic symbols for the inverter are shown in Fig.(3-1), shows the distinctive shape symbols.

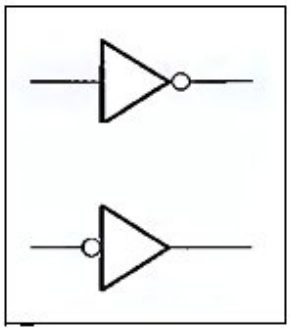


Fig.(3-1) Logic symbol for the inverter.

The negation indicator is a "bubble" (o) that indicates inversion or complementation when it appears on the input or output of any logic element. Generally, input is on the left of a logic symbol and the output is on the right. When appearing on the input, the bubble means that a 0 is the active, and the input is called an active-LOW input. When appearing on the output, the bubble means that a 0 is the active, and the output is called an active-LOW output.

When a HIGH level is applied to an inverter input, a LOW level will appear on its output. When a LOW level is applied to its input, a HIGH will appear on its output. This operation is summarized in Table 3-1, which shows the output for each possible input in terms of levels and corresponding bits. A table such as this is called a truth table.

Table 3–1 Inverter Truth Table.

INPUT	OUTPUT
LOW (0)	HIGH (1)
HIGH (1)	LOW (0)

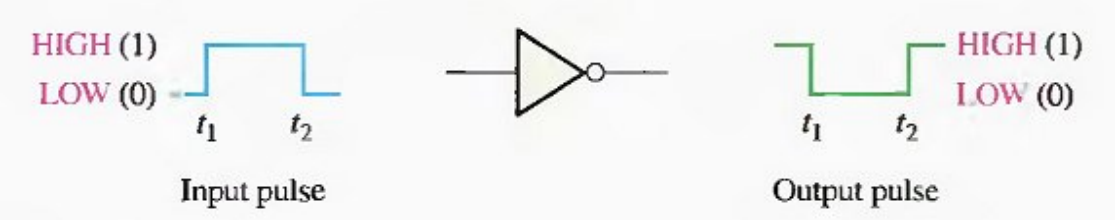


Fig.(3-2) Inverter operation.

The operation of an inverter (NOT circuit) can be expressed as follows: If the input variable is called A and the output variable is called X, then

$$X = \overline{A}$$

This expression states that the output is the complement of the input, so if A = 0, then X = 1, and if A = 1, then X = 0.

The AND Gate

The term gate is used to describe a circuit that performs a basic logic operation. The AND gate is composed of two or more inputs and a single output, as indicated by the standard logic symbols shown in Fig.(3-3). Inputs are on the left, and the output is on the right in each symbol. Gates with two inputs are shown; however, an AND gate can have any number of inputs greater than one.

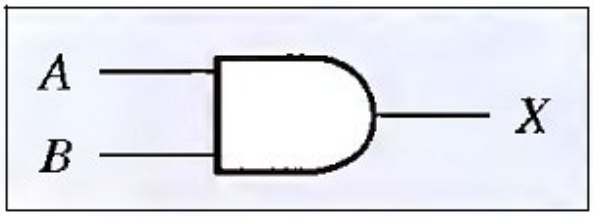


Fig.(3-3) AND gate symbol.

Operation of an AND Gate

An AND gate produces a HIGH output only when all of the inputs are HIGH. When any of the inputs is LOW, the output is LOW. Therefore, the basic purpose of an AND gate is to determine when certain conditions are simultaneously true, as indicated by HIGH levels on all of its inputs, and to produce a HIGH on its output to indicate that all these conditions are true. The inputs of the 2-input AND gate in Figure 3-8 are labeled A and B, and the output is labeled X. The gate operation can be stated as follows:

For a 2-input AND gate, output X is HIGH only when inputs A and B are HIGH; X is LOW when either A or B is LOW, or when both A and B are LOW.

Fig.(3-4) All possible logic levels for a 2-input AND gate.

The logical operation of a gate can be expressed with a truth table that lists all input combinations with the corresponding outputs, as illustrated in Table 3-2 for a 2-input AND gate. The truth table can be expanded to any number of inputs. For any AND gate, regardless of the number of inputs, the output is HIGH only when all inputs are HIGH.

Table 3-2 The truth table for a 2-input AND gate.

INPUTS		OUTPUT
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

1 = HIGH, 0 = LOW

The total number of possible combinations of binary inputs to a gate is determined by the following formula:

$$N = 2^n$$

where N is the number of possible input combinations and n is the number of input variables. To illustrate:

For two input variables: $N = 2^2 = 4$ combinations.
For three input variables: $N = 2^3 = 8$ combinations.
For four input variables: $N = 2^4 = 16$ combinations.

Q/ Develop the truth table for a 3-input AND gate.

Let's examine the waveform operation of an AND gate by looking at the inputs with respect to each other in order to determine the output level at any given time. In Fig.(3-10), inputs A and B are both HIGH (1) during the time interval, t_1 making output X HIGH (1) during this interval. During time interval t_2 input A is LOW (0) and input B is HIGH (1), so the output is LOW (0). During time interval t_3 , both inputs are HIGH (1), and therefore the output is HIGH (1). During time interval t_4 , input A is HIGH 0) and input B is LOW (0), resulting in a LOW (0) output. Finally, during time interval t_5 , input A is LOW (0), input B is LOW (0), and the output is therefore LOW (0). As you know, a diagram of input and output waveforms showing time relationships is called a timing diagram.

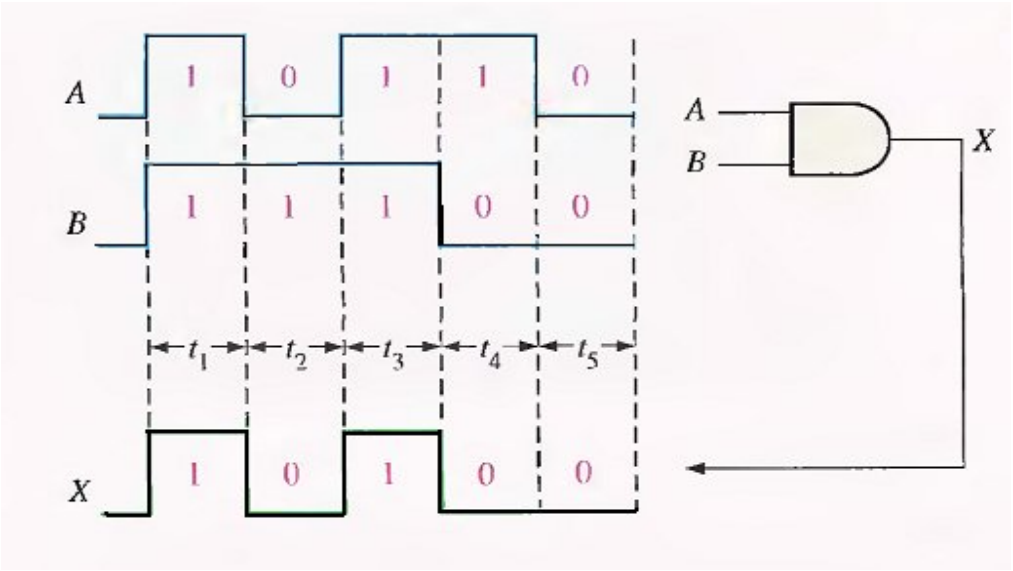


Fig.(3-5) Example of AND gate operation with a timing diagram showing input and output relationships.

Logic Expressions for an AND Gate

The logical AND function of two variables is represented mathematically either by placing a dot between the two variables, as A . B, or by simply writing the adjacent letters without the dot, as AB. We will normally use the latter notation because it is easier to write.

Boolean multiplication follows the same basic rules governing binary multiplication:

0 . 0 = 0
0 . 1 = 0
1 . 0 = 0
1 . 1 = 1

Boolean multiplication is the same as the AND function.

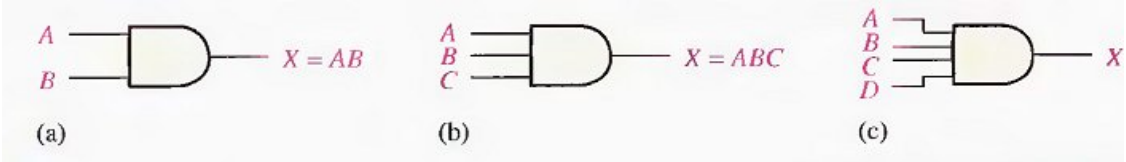


Fig.(3-6) Boolean expressions for AND gates with two, three, and four inputs.

The OR Gate

An OR gate can have more than two inputs. The OR gate is another of the basic gates from which all logic functions are constructed. An OR gate can have two or more inputs and performs what is known as logical addition.

An OR gate has two or more inputs and one output, as indicated by the standard logic symbol in Fig.(3-7), where OR gates with two inputs are illustrated. An OR gate can have any number of inputs greater than one.

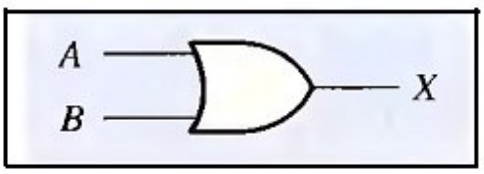


Fig.(3-7) Standard logic symbol for the OR gate.

Operation of an OR Gate

An OR gate produces a HIGH on the output when any of the inputs is HIGH. The output is LOW only when all of the inputs are LOW.

The inputs of the 2-input OR gate in Fig.(3-7) are labelled A and B. and the output is labelled X. The operation of the gate can be stated as follows:

For a 2-input OR gate, output X is HIGH when either input A or input B is HIGH, or when both A and B are HIGH; X is LOW only when both A and B are LOW.

The HIGH level is the active or asserted output level for the OR gate. Fig.(3-8) illustrates the operation for a 2-input OR gate for all four possible input combinations.

Fig.(3-8) All possible logic levels for a 2-input OR gate.

OR Gate Truth Table

The operation of a 2-input OR gate is described in Table 3-3. This truth table can be expanded for any number of inputs; but regardless of the number of inputs. the output is HIGH when one or more of the inputs are HIGH.

Table 3-3 The truth table for a 2-input OR gate.

INPUTS		OUTPUT
A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

1 = HIGH, 0 = LOW

Operation with Waveform Inputs

Now let's look at the operation of an OR gate with pulse waveform inputs, keeping in mind its logical operation. Again, the important thing in the analysis of gate operation with pulse waveforms is the time relationship of all the waveforms involved. For example, in Fig.(3-9), inputs A and B are both HIGH (1) during time interval t_1 making output X HIGH (1). During time interval t_2, input A is LOW (0), but because input B is HIGH (1), the output is HIGH (1). Both inputs are LOW (0) during time interval t_3 , so there is a LOW (0) output during this time. During time interval t_4 , the output is HIGH (1) because input A is HIGH (1).

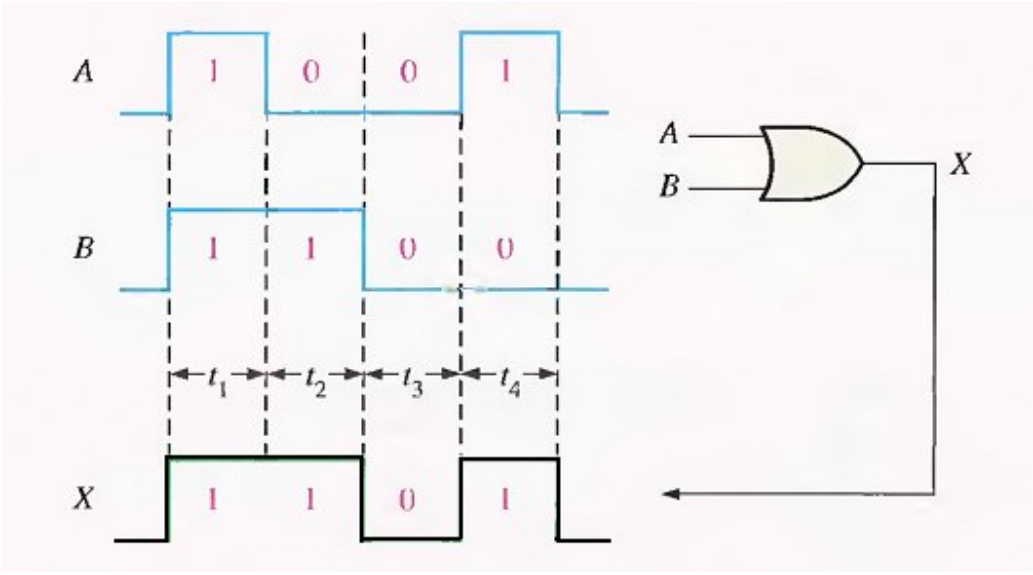


Fig.(3-9)) Example of OR gate operation with a timing diagram showing input and output relationships.

Logic Expressions for an OR Gate

The logical OR function of two variables is represented mathematically by a + between the two variables, for example, A + B. Addition in Boolean algebra involves variables whose values are either binary 1 or binary 0. The basic rules for Boolean addition are as follows:

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 1

Boolean addition is the same as the OR function.

Notice that Boolean addition differs from binary addition in the case where two 1 s are added. There is no carry in Boolean addition.

The operation of a 2-input OR gate can be expressed as follows: If one input variable is A, if the other input variable is B, and if the output variable is X, then the Boolean expression is

X=A+B

Fig.(3-10)(a) shows the OR gate logic symbol with two input variables and the output variable labelled.

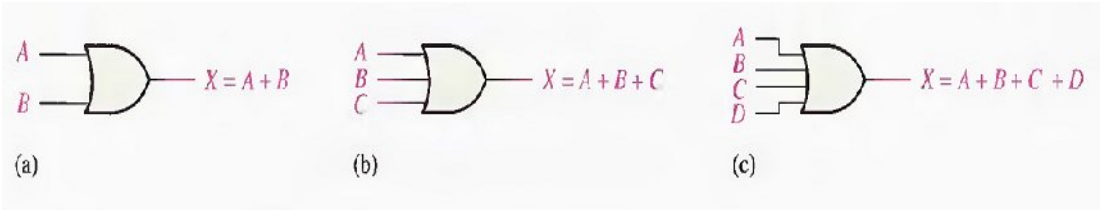


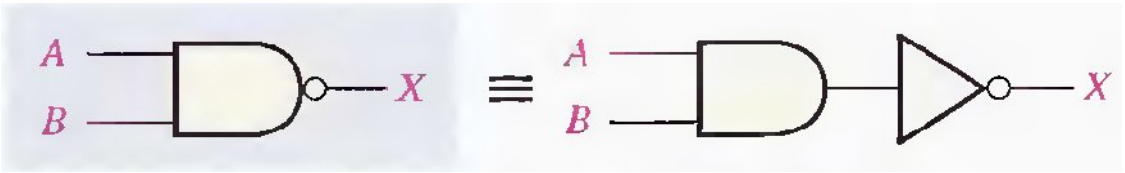
Fig.(3-10) Boolean expressions for AND gates with two, three, and four inputs.

To extend the OR expression to more than two input variables. a new letter is used for each additional variable. For instance, the function of a 3-input OR gate can be expressed as X = A + B + C. The expression for a 4-input OR gate can be written as X = A + B + C + D, and so on. Parts (b) and (c) of Fig.(3-10) how OR gates with three and four input variables, respectively.

THE NAND GATE

The NAND gate is a popular logic element because it can be used as a universal gate: that is, NAND gates can be used in combination to perform the AND, OR, and inverter operations.

The term NAND is a contraction of NOT-AND and implies an AND function with a complemented (inverted) output. The standard logic symbol for a 2-input NAND gate and its equivalency to an AND gate followed by an inverter are shown in Fig.(3-11)(a), where the symbol ≡ means equivalent to. A rectangular outline symbol is shown in part (b).



(a) (b)

Fig.(3-11) Standard NAND gate logic symbols.

Operation of a NAND Gate

A NAND gate produces a LOW output only when all the inputs are HIGH. When any of the inputs is LOW, the output will be HIGH. For the specific case of a 2-input NAND gate, as shown in Fig.(3-11) with the inputs labelled A and B and the output labelled X, the operation can be stated as follows:

For a 2-input NAND gate, output X is LOW only when inputs A and B are HIGH; X is HIGH when either A or B is LOW, or when both A and B are LOW.

Note that this operation is opposite that of the AND in terms of the output level. In a NAND gate, the LOW level (0) is the active or asserted output level, as indicated by the bubble on the output. Fig.(3-12) illustrates the operation of a 2-input NAND gate for all four input combinations, and Table 3-4 is the truth table summarizing the logical operation of the 2-input NAND gate.

The NAND is the same as the AND except the output is inverted.

Fig.(3-12) Operation of a 2-input NAND gate.

Table 3-4 Truth table for a 2-input NAND gate.

INPUTS		OUTPUT
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

1 = HIGH, 0 = LOW.

Negative-OR Equivalent Operation of a NAND Gate

Inherent in a NAND gate's operation is the fact that one or more LOW inputs produce a HIGH output. Table 3-4 shows that output X is HIGH (1) when any of the inputs, A and B, is LOW (0). From this viewpoint, a NAND gate can be used for an OR operation that requires one or more LOW inputs to produce a HIGH output. This aspect of NAND operation is referred to as negative-OR. The term negative in this context mean that the inputs are defined to be in the active or asserted state when LOW.

For a 2-input NAND gate performing a negative-OR operation, output X is HIGH when either input A or input B is LOW, or when both A and B are LOW.

When a NAND gate is used to detect one or more LOWs on its inputs rather than all HIGHs, it is performing the negative-OR operation and is represented by the standard logic symbol shown in Fig.(3-13).

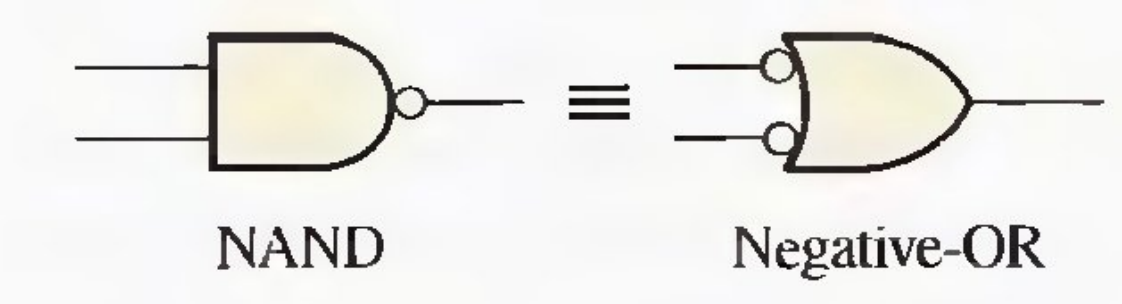


Fig.(3-13) Standard symbols representing the two equivalent operation of a NAND gate.

Logic Expressions for a NAND Gate

The Boolean expression for the output of a 2-input NAND gate is

$$X = \overline{AB}$$

This expression says that the two input variables, A and B, are first ANDed and then complemented, as indicated by the bar over the AND expression. This is a description in equation form of the operation of a NAND gate with two inputs. Evaluating this expression for all possible values of the two input variables, you get the results shown in Table 3-5.

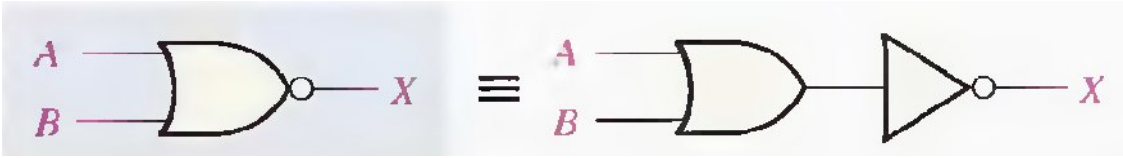
Table 3-5.

A	B	$\overline{AB} = X$
0	0	$\overline{0 \cdot 0} = \overline{0} = 1$
0	1	$\overline{0 \cdot 1} = \overline{0} = 1$
1	0	$\overline{1 \cdot 0} = \overline{0} = 1$
1	1	$\overline{1 \cdot 1} = \overline{1} = 0$

The NOR Gate

The NOR gate, like the NAND gate, is a useful logic element because it can also be used as a universal gate; that is, NOR gates can be used in combination to perform the AND, OR, and inverter operations.

The term NOR is a contraction of NOT-OR and implies an OR function with an inverted (complemented) output. The standard logic symbol for a 2-input NOR gate and its equivalent OR gate followed by an inverter are shown in Fig.(3-14)(a).



(a) (b)

Fig.(3-14) Standard NOR gate logic symbols.

Operation of a NOR Gate

A NOR gate produces a LOW output when any of its inputs is HIGH. Only when all of its inputs are LOW is the output HIGH. For the specific case of a 2-input NOR gate, as shown in Fig.(3-14) with the inputs labelled A and B and the output labelled X, the operation can be stated as follows:

For a 2-input NOR gate, output X is LOW when either input A or input B is HIGH, or when both A and B are HIGH; X is HIGH only when both A and B are LOW.

This operation results in an output level opposite that of the OR gate. In a NOR gate, the LOW output is the active or asserted output level as indicated by the bubble on the output.

Fig.(3-15) illustrates the operation of a 2-input NOR gate for all four possible input combinations, and Table 3-6 is the truth table for a 2-input NOR gate.

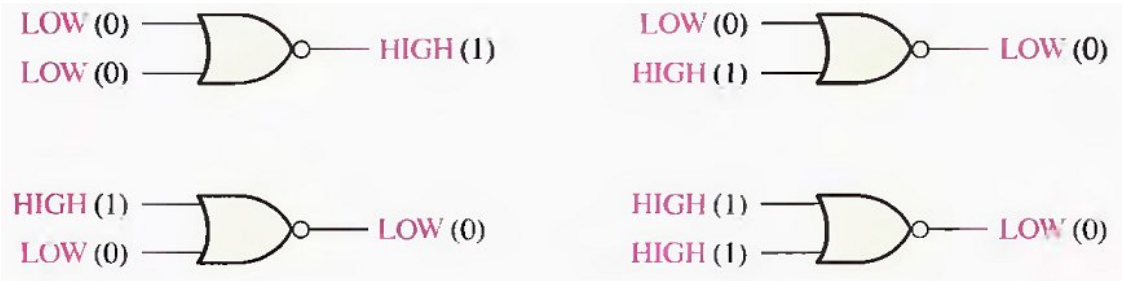


Fig.(3-15) Operation of a 2-input NOR gate.

Table 3-6 Truth table for a 2-input NOR gate.

INPUTS		OUTPUT
A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

1 = HIGH, 0 = LOW.

Negative-AND Equivalent Operation of the NOR Gate

A NOR gate, like the NAND, has another aspect of its operation that is inherent in the way it logically functions. Table 3-6 shows that a HIGH is produced on the gate output only when all of the inputs are LOW. From this viewpoint, a NOR gate can be used for an AND operation that requires all LOW inputs to produce a HIGH output. This aspect of NOR operation is called negative-AND.

The term negative in this context means that the inputs are defined to be in the active or asserted state when LOW.

For a 2-input NOR gate performing a negative-AND operation, output X is HIGH only when both inputs A and B are LOW.

When a NOR gate is used to detect all LOWs on its inputs rather than one or more HIGHs, it is performing the negative-AND operation and is represented by the standard symbol in Fig.(3-16). It is important to remember that the two symbols in Fig.(3-16), represent the same physical gate and serve only to distinguish between the two modes of its operation.

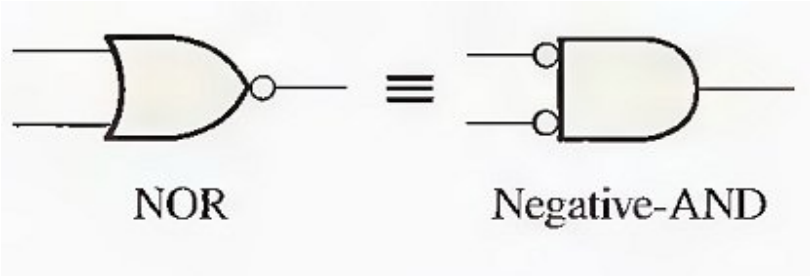


Fig.(3-16) Standard symbols representing the two equivalent operations of a NOR gate.

Logic Expressions for a NOR Gate

The Boolean expression for the output of a 2-input NOR gate can be written as

$$X = \overline{A+B}$$

This equation says that the two input variables are first ORed and then complemented, as indicated by the bar over the OR expression. Evaluating this expression, you get the results shown in Table 3-7. The NOR expression can be extended to more than two input variables by including additional letters to represent the other variables.

Table 3-7

A	B	$\overline{A+B}=X$
0	0	$\overline{0+0}=\overline{0}=1$
0	1	$\overline{0+1}=\overline{1}=0$
1	0	$\overline{1+0}=\overline{1}=0$
1	1	$\overline{1+1}=\overline{1}=0$

THE EXCLUSIVE-OR AND EXCLUSIVE-NOR GATES

Exclusive-OR and exclusive-NOR gates are formed by a combination of other gates already discussed. However, because of their fundamental importance in many applications, these gates are often treated as basic logic elements with their own unique symbols.

The Exclusive-OR Gate

Standard symbol for an exclusive-OR (XOR for short) gate is shown in Fig.(3-17). The XOR gate has only two inputs.

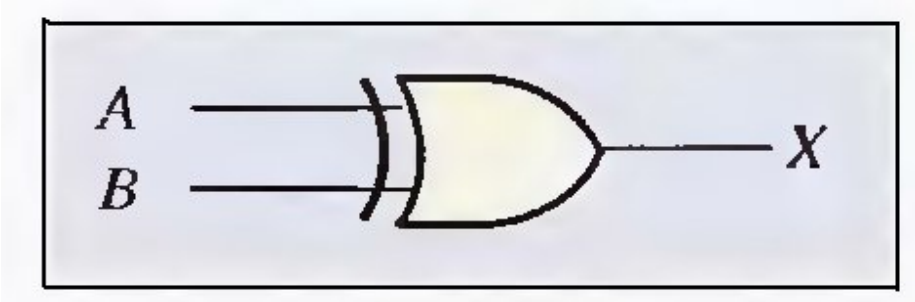


Fig.(3-17) Standard symbol for an exclusive-OR.

For an exclusive-OR gate, output X is HIGH when input A is LOW and input B is HIGH, or when input A is HIGH and input B is LOW: X is LOW when A and B are both HIGH or both LOW.

The four possible input combinations and the resulting outputs for an XOR gate are illustrated in Fig.(3-18). The HIGH level is the active or asserted output level and occurs only when the inputs are at opposite levels. The operation of an XOR gate is summarized in the table shown in Table 3-8.

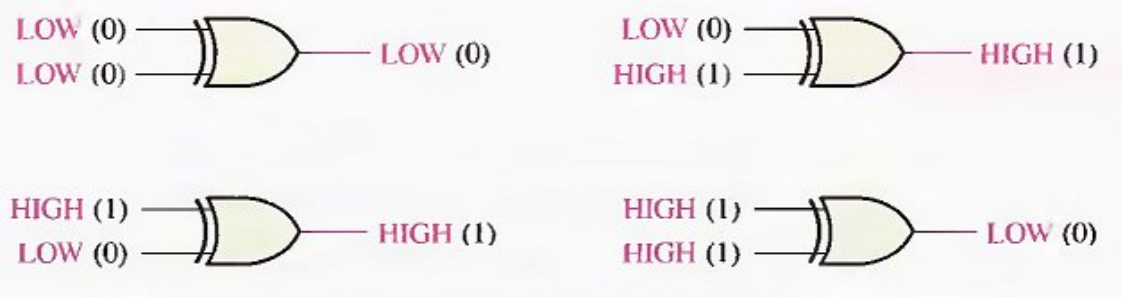


Fig.(3-18) All possible logic levels for an exclusive-OR gate.

Table 3-8 Truth table for an exclusive-OR gate.

INPUTS		OUTPUT
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

The Exclusive-NOR Gate

Standard symbols for an exclusive-NOR (XNOR) gate are shown in Fig.(3-19). Like the XOR gate, an XNOR has only two inputs. The bubble on the output of the XNOR symbol indicates that its output is opposite that of the XOR gate. When the two input logic levels are opposite, the output of the exclusive-NOR gate is LOW. The operation can be stated as follows (A and B are inputs, X is the output):

For an exclusive-NOR gate, output X is LOW when input A is LOW and input B is HIGH, or when A is HIGH and B is LOW; X is HIGH when A and B are both HIGH or both LOW.

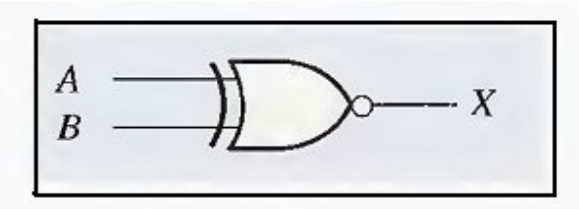


Fig.(3-19) Standard logic symbols for the exclusive-NOR gate.

The four possible input combinations and the resulting outputs for an XNOR gate are shown in Fig,(3-20). The operation of an XNOR gate is summarized in Table 3-9. Notice that the output is HIGH when the same level is on both inputs.

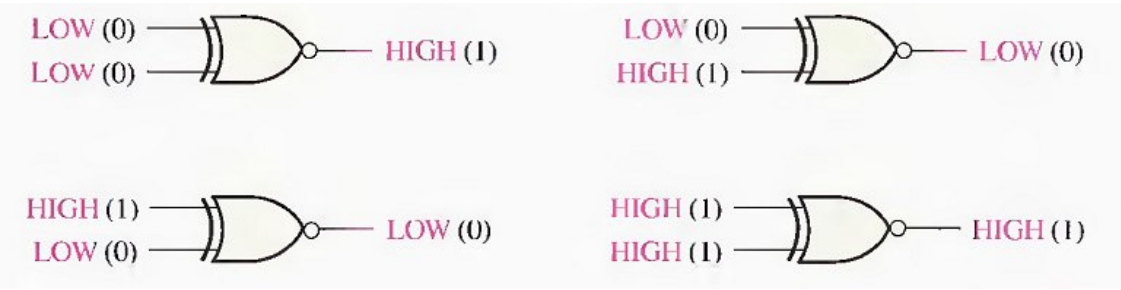


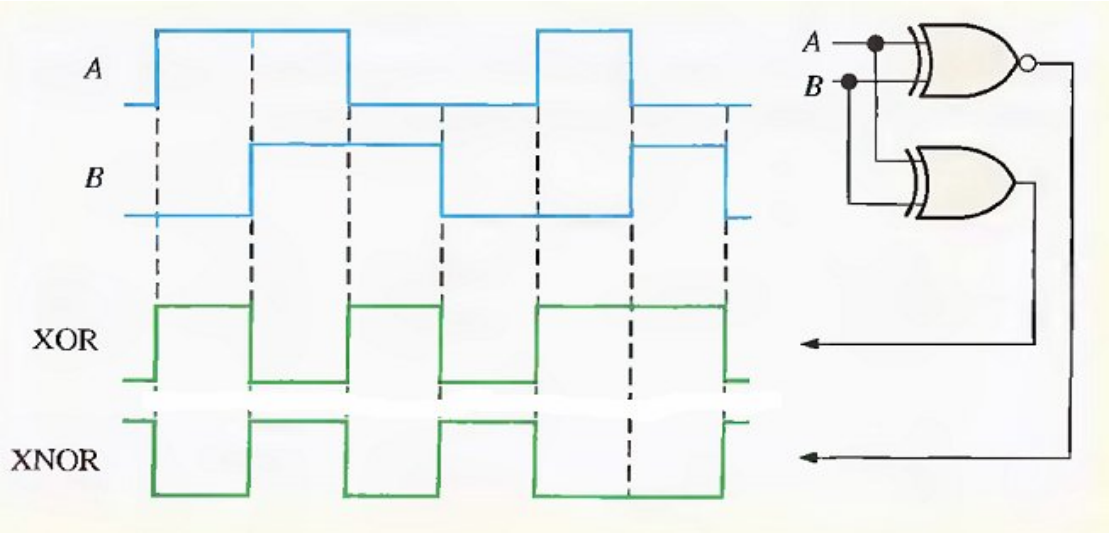
Fig.(3-20) All possible logic levels for an exclusive-NOR gate.

Table 3-9 Truth table for an exclusive-NOR gate.

INPUTS		OUTPUT
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

Example

Determine the output waveforms for the XOR gate and for the XNOR gate, given the input waveforms, A and B, in Figure below:



4 BOOLEAN ALGEBRA AND LOGIC SIMPLIFICATION

BOOLEAN OPERATIONS AND EXPRESSIONS

Variable, complement, and literal are terms used in Boolean algebra. A variable is a symbol used to represent a logical quantity. Any single variable can have a 1 or a 0 value. The complement is the inverse of a variable and is indicated by a bar over variable (overbar). For example, the complement of the variable A is $\overline{A}$. If A = 1, then $\overline{A}$ = 0. If A = 0, then $\overline{A}$ = 1. The complement of the variable A is read as "not A" or "A bar." Sometimes a prime symbol rather than an overbar is used to denote the complement of a variable; for example, B' indicates the complement of B. A literal is a variable or the complement of a variable.

Boolean Addition

Recall from part 3 that Boolean addition is equivalent to the OR operation. In Boolean algebra, a sum term is a sum of literals. In logic circuits, a sum term is produced by an OR operation with no AND operations involved. Some examples of sum terms are $A + B$, $A + \overline{B}$, $A + B + \overline{C}$, and $\overline{A} + B + C + \overline{D}$.

A sum term is equal to 1 when one or more of the literals in the term are 1. A sum term is equal to 0 only if each of the literals is 0.

Example

Determine the values of A, B, C, and D that make the sum term $A + \overline{B} + C + \overline{D}$ equal to 0.

Boolean Multiplication

Also recall from part 3 that Boolean multiplication is equivalent to the AND operation. In Boolean algebra, a product term is the product of literals. In logic circuits, a product term is produced by an AND operation with no OR operations involved. Some examples of product terms are AB, $A\overline{B}$, ABC, and $\overline{A}BC\overline{D}$.

A product term is equal to 1 only if each of the literals in the term is 1. A product term is equal to 0 when one or more of the literals are 0.

Example

Determine the values of A, B, C, and D that make the product term $\overline{A}BC\overline{D}$ equal to 1.

LAWS AND RULES OF BOOLEAN ALGEBRA

■ *Laws of Boolean Algebra*

The basic laws of Boolean algebra-the commutative laws for addition and multiplication, the associative laws for addition and multiplication, and the distributive law-are the same as in ordinary algebra.

Commutative Laws

►The commutative law of addition for two variables is written as

$A+B = B+A$

This law states that the order in which the variables are ORed makes no difference. Remember, in Boolean algebra as applied to logic circuits, addition and the OR operation are the same. Fig.(4-1) illustrates the commutative law as applied to the OR gate and shows that it doesn't matter to which input each variable is applied. (The symbol ≡ means "equivalent to.").

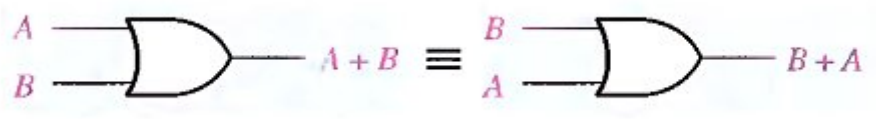


Fig.(4-1) Application of commutative law of addition.

►The commutative law of multiplication for two variables is

A.B = B.A

This law states that the order in which the variables are ANDed makes no difference. Fig.(4-2), il1ustrates this law as applied to the AND gate.

Fig.(4-2) Application of commutative law of multiplication.

Associative Laws :

►The associative law of addition is written as follows for three variables:

A + (B + C) = (A + B) + C

This law states that when ORing more than two variables, the result is the same regardless of the grouping of the variables. Fig.(4-3), illustrates this law as applied to 2-input OR gates.

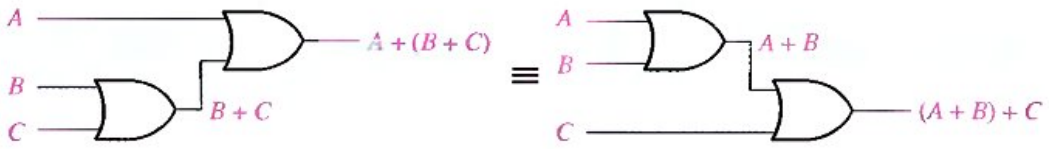


Fig.(4-3) Application of associative law of addition.

►The associative law of multiplication is written as follows for three variables:

A(BC) = (AB)C

This law states that it makes no difference in what order the variables are grouped when ANDing more than two variables. Fig.(4-4) illustrates this law as applied to 2-input AND gates.

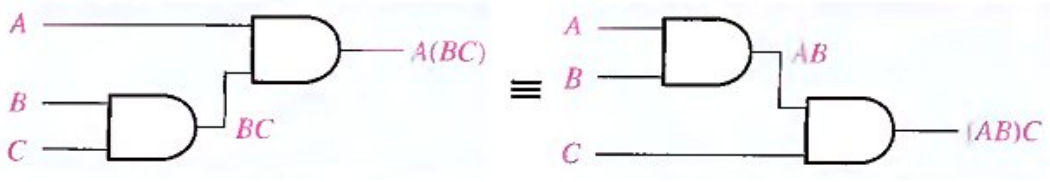


Fig.(4-4) Application of associative law of multiplication.

Distributive Law:

►The distributive law is written for three variables as follows:

$$A(B + C) = AB + AC$$

This law states that ORing two or more variables and then ANDing the result with a single variable is equivalent to ANDing the single variable with each of the two or more variables and then ORing the products. The distributive law also expresses the process of factoring in which the common variable A is factored out of the product terms, for example,

$$AB + AC = A(B + C).$$

Fig.(4-5) illustrates the distributive law in terms of gate implementation.

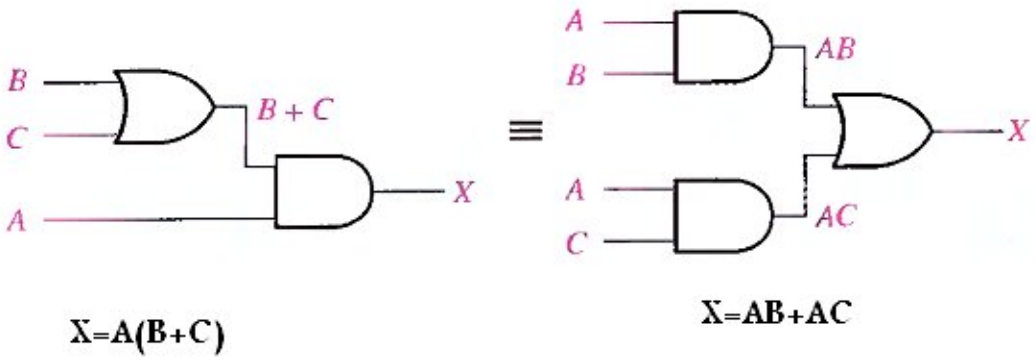


Fig.(4-5) Application of distributive law.

■ ***Rules of Boolean Algebra***

Table 4-1 lists 12 basic rules that are useful in manipulating and simplifying Boolean expressions. Rules 1 through 9 will be viewed in terms of their application to logic gates. Rules 10 through 12 will be derived in terms of the simpler rules and the laws previously discussed.

Table 4-1 Basic rules of Boolean algebra.

1. $A + 0 = A$	**7.** $A \cdot A = A$
2. $A + 1 = 1$	**8.** $A \cdot \overline{A} = 0$
3. $A \cdot 0 = 0$	**9.** $\overline{\overline{A}} = A$
4. $A \cdot 1 = A$	**10.** $A + AB = A$
5. $A + A = A$	**11.** $A + \overline{A}B = A + B$
6. $A + \overline{A} = 1$	**12.** $(A + B)(A + C) = A + BC$

A, B, or *C* can represent a single variable or a combination of variables.

Rule 1. A + 0 = A

A variable ORed with 0 is always equal to the variable. If the input variable A is 1, the output variable X is 1, which is equal to A. If A is 0, the output is 0, which is also equal to A. This rule is illustrated in Fig.(4-6), where the lower input is fixed at 0.

A+0=A

Fig.(4-6)

Rule 2. A + 1 = 1

A variable ORed with 1 is always equal to 1. A 1 on an input to an OR gate produces a 1 on the output, regardless of the value of the variable on the other input. This rule is illustrated in Fig.(4-7), where the lower input is fixed at 1.

X=A+1=1

Fig.(4-7)

Rule 3. A . 0 = 0

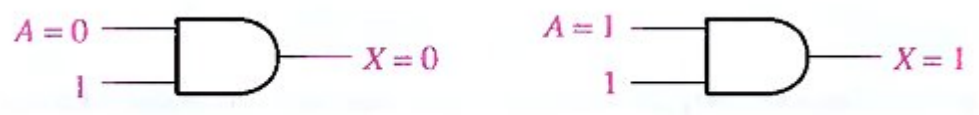
A variable ANDed with 0 is always equal to 0. Any time one input to an AND gate is 0, the output is 0, regardless of the value of the variable on the other input. This rule is illustrated in Fig.(4-8), where the lower input is fixed at 0.

X= A.0 = 0

Fig.(4-8)

Rule 4. A . 1 = A

A variable ANDed with 1 is always equal to the variable. If A is 0 the output of the AND gate is 0. If A is 1, the output of the AND gate is 1 because both inputs are now 1s. This rule is shown in Fig.(4-9), where the lower input is fixed at 1.

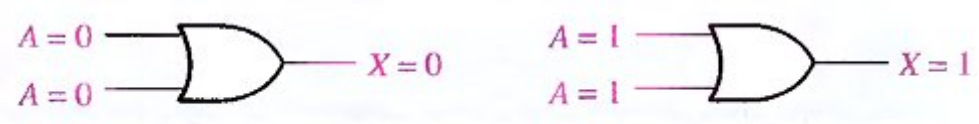


X = A . 1 = A

Fig.(4-9)

Rule 5. A + A = A

A variable ORed with itself is always equal to the variable. If A is 0, then 0 + 0 = 0; and if A is 1, then 1 + 1 = 1. This is shown in Fig.(4-10), where both inputs are the same variable.



X = A + A = A

Fig.(4-10)

Rule 6. A + $\overline{A}$ = 1

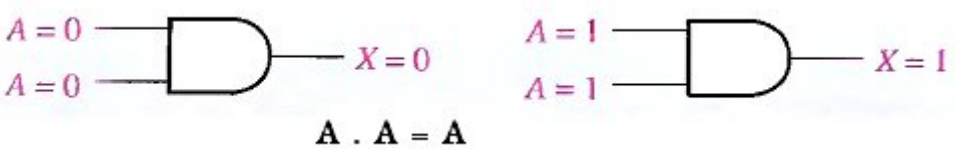
A variable ORed with its complement is always equal to 1. If A is 0, then 0 + $\overline{0}$ = 0 + 1 = 1. If A is l, then 1 + 1 = 1+ 0 = 1. See Fig.(4-11), where one input is the complement of the other.

X = A + $\overline{A}$ = 1

Fig.(4-11)

Rule 7. A . A = A

A variable ANDed with itself is always equal to the variable. If A = 0, then 0.0 = 0; and if A = 1. then 1.1 = 1. Fig.(4-12) illustrates this rule.



A . A = A

Fig.(4-12)

Rule 8. $A \cdot \overline{A} = 0$

A variable ANDed with its complement is always equal to 0. Either A or $\overline{A}$ will always be 0: and when a 0 is applied to the input of an AND gate. the output will be 0 also. Fig.(4-13) illustrates this rule.

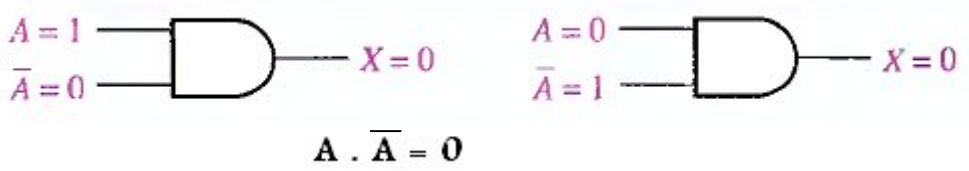


Fig.(4-13)

Rule 9 $A = \overline{\overline{A}}$

The double complement of a variable is always equal to the variable. If you start with the variable A and complement (invert) it once, you get $\overline{A}$. If you then take $\overline{A}$ and complement (invert) it, you get A, which is the original variable. This rule is shown in Fig.(4-14) using inverters.

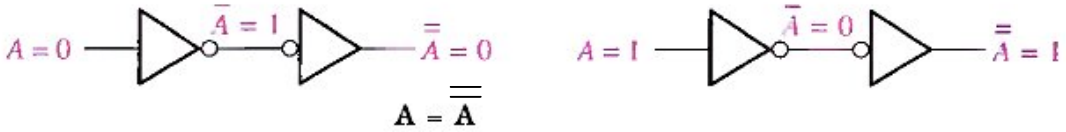


Fig.(4-14)

Rule 10. $A + AB = A$

This rule can be proved by applying the distributive law, rule 2, and rule 4 as follows:

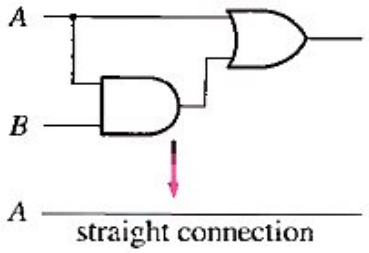
$A + AB = A(1 + B)$	Factoring (distributive law)
$= A \cdot 1$	Rule 2: $(1 + B) = 1$
$= A$	Rule 4: $A \cdot 1 = A$

The proof is shown in Table 4-2, which shows the truth table and the resulting logic circuit simplification.

Table 4-2

A	B	AB	A + AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

equal



Rule 11. $A + \overline{A}B = A + B$

This rule can be proved as follows:

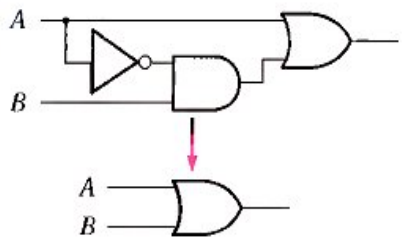
$A + \overline{A}B = (A + AB) + \overline{A}B$	Rule 10: $A = A + AB$
$= (AA + AB) + \overline{A}B$	Rule 7: $A = AA$
$= AA + AB + A\overline{A} + \overline{A}B$	Rule 8: adding $A\overline{A} = 0$
$= (A + \overline{A})(A + B)$	Factoring
$= 1.\ (A + B)$	Rule 6: $A + \overline{A} = 1$
$= A + B$	Rule 4: drop the 1

The proof is shown in Table 4-3, which shows the truth table and the resulting logic circuit simplification.

Table 4-3

A	B	$\overline{A}B$	$A + \overline{A}B$	A + B
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

equal



Rule 12. $(A + B)(A + C) = A + BC$

This rule can be proved as follows:

$(A + B)(A + C) = AA + AC + AB + BC$		Distributive law
$= A + AC + AB + BC$		Rule 7: $AA = A$
$= A(1 + C) + AB + BC$		Rule 2: $1 + C = 1$
$= A. 1 + AB + BC$		Factoring (distributive law)
$= A(1 + B) + BC$		Rule 2: $1 + B = 1$
$= A. 1 + BC$		Rule 4: $A . 1 = A$
$= A + BC$		

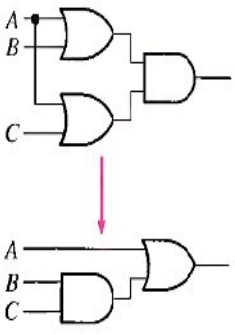
The proof is shown in Table 4-4, which shows the truth table and the resulting logic circuit

simplification.

Table 4-4

A	B	C	A + B	A + C	(A + B)(A + C)	BC	A + BC
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	0	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

equal



DEMORGAN'S THEOREMS

DeMorgan, a mathematician who knew Boole, proposed two theorems that are an important part of Boolean algebra. In practical terms. DeMorgan's theorems provide mathematical verification of the equivalency of the NAND and negative-OR gates and the equivalency of the NOR and negative-AND gates, which were discussed in part 3.

One of DeMorgan's theorems is stated as follows:

The complement of a product of variables is equal to the sum of the complements of the variables,

Stated another way,

The complement of two or more ANDed variables is equivalent to the OR of the complements of the individual variables.

The formula for expressing this theorem for two variables is

$$\overline{XY} = \overline{X} + \overline{Y}$$

DeMorgan's second theorem is stated as follows:

The complement of a sum of variables is equal to the product of the complements of the variables.

Stated another way,

The complement of two or more ORed variables is equivalent to the AND of the complements of the individual variables,

The formula for expressing this theorem for two variables is

$$\overline{X + Y} = \overline{X}\,\overline{Y}$$

Fig.(4-15) shows the gate equivalencies and truth tables for the two equations above.

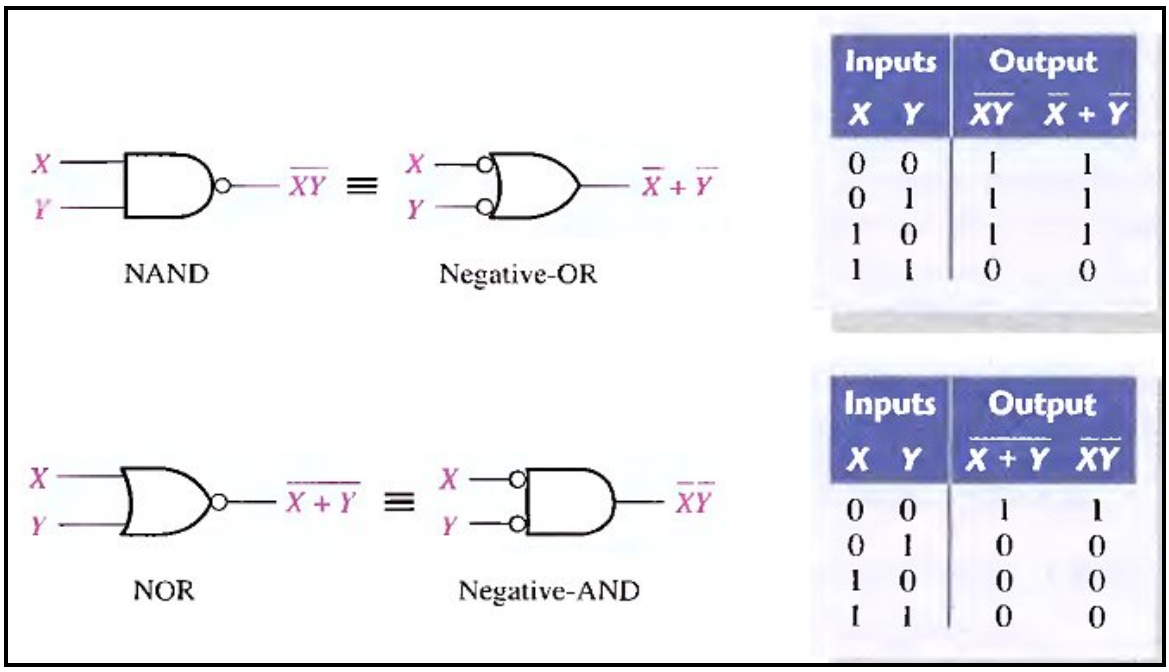


Fig.(4-15) Gate equivalencies and the corresponding truth tables that illustrate DeMorgan's theorems.

As stated, DeMorgan's theorems also apply to expressions in which there are more than two variables. The following examples illustrate the application of DeMorgan's theorems to 3-variable and 4-variable expressions.

Example

Apply DeMorgan's theorems to the expressions $\overline{XYZ}$ and $\overline{X + Y + z}$.

$$\overline{XYZ} = \overline{X} + \overline{Y} + \overline{Z}$$

$$\overline{X + y + Z} = \overline{X}\,\overline{Y}\,\overline{Z}$$

Example

Apply DeMorgan's theorems to the expressions $\overline{WXYZ}$ and $\overline{W + X + y + z}$.

$$WXYZ = \overline{W} + \overline{X} + \overline{y} + \overline{Z}$$

$$\overline{W + X + y + Z} = \overline{W}\,\overline{X}\,\overline{Y}\,\overline{Z}$$

Applying DeMorgan's Theorems

The following procedure illustrates the application of DeMorgan's theorems and Boolean algebra to the specific expression

$$\overline{\overline{A + B\overline{C}} + D(\overline{E + F})}$$

Step 1. Identify the terms to which you can apply DeMorgan's theorems, and think of each term as a single variable. Let $\overline{A + B\overline{C}} = X$ and $D\overline{(E + F)} = Y$.

Step 2. Since $\overline{X + Y} = \overline{X}\,\overline{Y}$,

$$\overline{\overline{A + B\overline{C}} + D(\overline{E + F})} = \overline{\overline{A + B\overline{C}}}\ \overline{D(\overline{E + F})}$$

Step 3. Use rule 9 ($A = \overline{\overline{A}}$) to cancel the double bars over the left term (this is not part of DeMorgan's theorem).

$$\overline{\overline{A + B\overline{C}}}\ \overline{D(\overline{E + F})} = (A + B\overline{C})\ \overline{D(\overline{E + F})}$$

Step 4. Applying DeMorgan's theorem to the second term,

$$(A + B\overline{C})\overline{D(\overline{E + F})} = (A + B\overline{C})(\overline{D} + \overline{(E + F)})$$

Step 5. Use rule 9 ($A = \overline{\overline{A}}$) to cancel the double bars over the $E + \overline{F}$ part of the term.

$$(A + B\overline{C})(\overline{D} + \overline{\overline{E + F}}) = (A + B\overline{C})(\overline{D} + E + \overline{F})$$

Example

Apply DeMorgan's theorems to each of the following expressions:

(a) $\overline{(A + B + C)D}$ (b) $\overline{ABC + DEF}$ (c) $\overline{A\overline{B} + C\overline{D} + EF}$

Example

The Boolean expression for an exclusive-OR gate is $A\overline{B} + \overline{A}B$. With this as a starting point, use DeMorgan's theorems and any other rules or laws that are applicable to develop an expression for the exclusive-NOR gate.

BOOLEAN ANALYSIS OF LOGIC CIRCUITS

Boolean algebra provides a concise way to express the operation of a logic circuit formed by a combination of logic gates so that the output can be determined for various combinations of input values.

Boolean Expression for a Logic Circuit

To derive the Boolean expression for a given logic circuit, begin at the left-most inputs and work toward the final output, writing the expression for each gate. For the example circuit in Fig.(4-16), the Boolean expression is determined as follows:

- The expression for the left-most AND gate with inputs C and D is CD.
- The output of the left-most AND gate is one of the inputs to the OR gate and B is the other input. Therefore, the expression for the OR gate is B + CD.
- The output of the OR gate is one of the inputs to the right-most AND gate and A is the other input. Therefore, the expression for this AND gate is A(B + CD), which is the final output expression for the entire circuit.

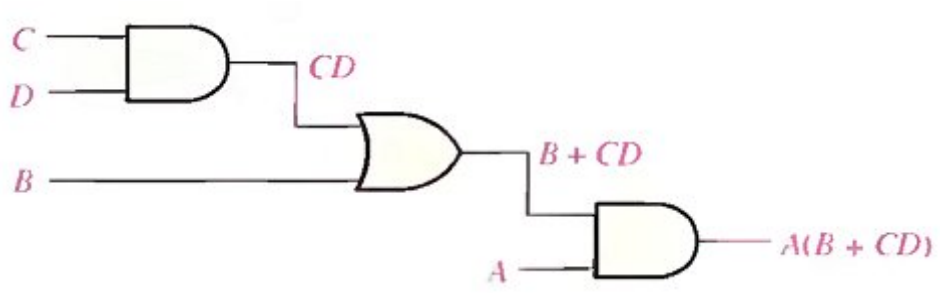


Fig.(4-16) A logic circuit showing the development of the Boolean expression for the output.

Constructing a Truth Table for a Logic Circuit

Once the Boolean expression for a given logic circuit has been determined, a truth table that shows the output for all possible values of the input variables can be developed. The procedure requires that you evaluate the Boolean expression for all possible combinations of values for the input variables. In the case of the circuit in Fig.(4-16), there are four input variables (A, B, C, and D) and therefore sixteen ($2^4 = 16$) combinations of values are possible.

Putting the Results in Truth Table format

The first step is to list the sixteen input variable combinations of 1s and 0s in a binary sequence as shown in Table 4-5. Next, place a 1 in the output column for each combination of input variables that was determined in the evaluation. Finally, place a 0 in the output column for all other combinations of input variables. These results are shown in the truth table in Table 4-5.

Table 4-5

INPUTS				OUTPUT
A	B	C	D	A(B + CD)
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

SIMPLIFICATION USING BOOLEAN ALGEBRA

A simplified Boolean expression uses the fewest gates possible to implement a given expression.

Example

Using Boolean algebra techniques, simplify this expression:

$$AB + A(B + C) + B(B + C)$$

Solution

Step 1: Apply the distributive law to the second and third terms in the expression, as follows:

$$AB + AB + AC + BB + BC$$

Step 2: Apply rule 7 (BB = B) to the fourth term.

$$AB + AB + AC + B + BC$$

Step 3: Apply rule 5 (AB + AB = AB) to the first two terms.

$$AB + AC + B + BC$$

Step 4: Apply rule 10 (B + BC = B) to the last two terms.

AB + AC + B

Step 5: Apply rule 10 (AB + B = B) to the first and third terms.

B+AC

At this point the expression is simplified as much as possible.

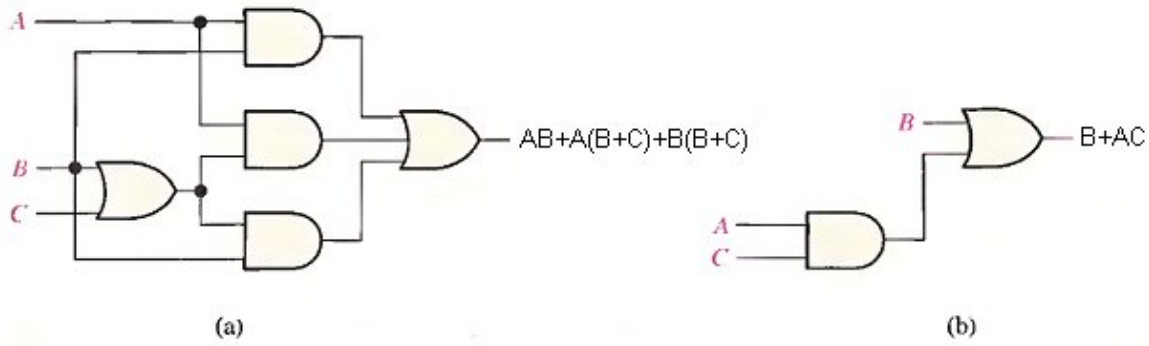


Fig.(4-17) Gate circuits for example above.

Example

Simplify the Boolean expressions:

1- $A\overline{B} + A(\overline{B + C}) + B(\overline{B + C})$.

2- $[A\overline{B}(C + BD) + \overline{A}\,\overline{B}]C$

3- $\overline{A}BC + A\overline{B}\,\overline{C} + \overline{A}\,\overline{B}\,\overline{C} + A\overline{B}C + ABC$

Standard and Canonical Forms

STANDARD FORMS OF BOOLEAN EXPRESSIONS

All Boolean expressions, regardless of their form, can be converted into either of two standard forms: the sum-of-products form or the product-of-sums form. Standardization makes the evaluation, simplification, and implementation of Boolean expressions much more systematic and easier.

The Sum-of-Products (SOP) Form

When two or more product terms are summed by Boolean addition, the resulting expression is a sum-of-products (SOP). Some examples are:

$AB + ABC$

$ABC + \overline{C}DE + \overline{B}C\overline{D}$

$AB + BCD + AC$

Also, an SOP expression can contain a single-variable term, as in

$A + AB\overline{C} + BC\overline{D}$.

In an SOP expression a single overbar cannot extend over more than one variable.

Example

Convert each of the following Boolean expressions to SOP form:

(a) $AB + B(CD + EF)$

(b) $(A + B)(B + C + D)$

(c) $\overline{\overline{(A + B)} + C}$

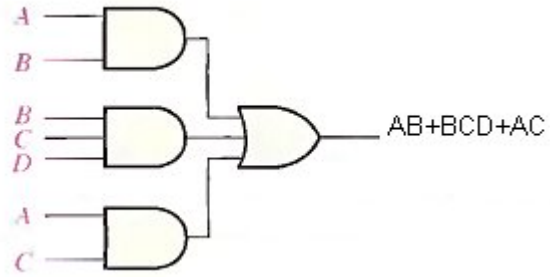


Fig.(4-18) Implementation of the SOP expression AB + BCD + AC.

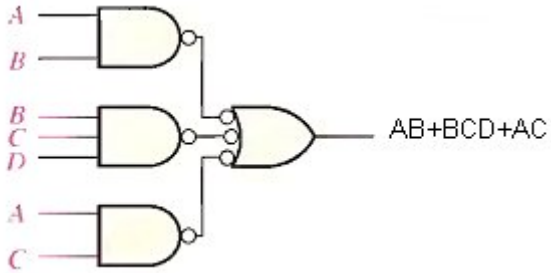


Fig.(4-19) This NAND/NAND implementation is equivalent to the AND/OR in figure above.

The Standard SOP Form

So far, you have seen SOP expressions in which some of the product terms do not contain all of the variables in the domain of the expression. For example, the expression $A\overline{B}\overline{C} + A\overline{B}D + AB\overline{C}\overline{D}$ has a domain made up of the variables A, B, C. and D. However, notice that the complete set of variables in the domain is not represented in the first two terms of the expression; that is, D or $\overline{D}$ is missing from the first term and C or $\overline{C}$ is missing from the second term.

A standard SOP expression is one in which all the variables in the domain appear in each product term in the expression. For example, $\overline{A}BC\overline{D} + AB\overline{C}D + A\overline{B}CD$ is a standard SOP expression.

Converting Product Terms to Standard SOP:

Each product term in an SOP expression that does not contain all the variables in the domain can be expanded to standard SOP to include all variables in the domain and their complements. As stated in the following steps, a nonstandard SOP expression is converted into standard form using Boolean algebra rule 6 ($A + \overline{A} = 1$) from Table 4-1: A variable added to its complement equals 1.

Step 1. Multiply each nonstandard product term by a term made up of the sum of a missing variable and its complement. This results in two product terms. As you know, you can multiply anything by 1 without changing its value.

Step 2. Repeat Step 1 until all resulting product terms contain all variables in the domain in either complemented or uncomplemented form. In converting a product term to standard form, the number of product terms is doubled for each missing variable.

Example

Convert the following Boolean expression into standard SOP form:

$A\overline{B}C + \overline{A}\,\overline{B} + AB\overline{C}D$

Solution

The domain of this SOP expression A, B, C, D. Take one term at a time. The first term, $A\overline{B}C$, is missing variable D or $\overline{D}$, so multiply the first term by $(D + \overline{D})$ as follows:

$A\overline{B}C = A\overline{B}C(D + \overline{D}) = A\overline{B}CD + A\overline{B}C\overline{D}$

In this case, two standard product terms are the result.

The second term, $\overline{A}\,\overline{B}$, is missing variables C or $\overline{C}$ and D or $\overline{D}$, so first multiply the second term by $C + \overline{C}$ as follows:

$AB = \overline{A}\,\overline{B}(C + \overline{C}) = \overline{A}\,\overline{B}C + \overline{A}\,\overline{B}\,\overline{C}$

The two resulting terms are missing variable D or $\overline{D}$, so multiply both terms by $(D + \overline{D})$ as follows:

$\overline{A}\,\overline{B}C(D + \overline{D}) + \overline{A}\,\overline{B}\,\overline{C}(D + \overline{D})$

$= \overline{A}\,\overline{B}CD + \overline{A}\,\overline{B}C\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + \overline{A}\,\overline{B}\,\overline{C}\,\overline{D}$

In this case, four standard product terms are the result.

The third term, $AB\overline{C}D$, is already in standard form. The complete standard SOP form of the original expression is as follows:

$A\overline{B}C + \overline{A}\,\overline{B} + AB\overline{C}D = A\overline{B}CD + A\overline{B}C\overline{D} + \overline{A}\,\overline{B}CD + \overline{A}\,\overline{B}C\overline{D} + \overline{A}\,\overline{B}CD + \overline{A}\,\overline{B}\,\overline{C}\,\overline{D} + AB\overline{C}D$

The Product-of-Sums (POS) Form

A sum term was defined before as a term consisting of the sum (Boolean addition) of literals (variables or their complements). When two or more sum terms are multiplied, the resulting expression is a product-of-sums (POS). Some examples are

$(\overline{A} + B)(A + \overline{B} + C)$

$(A + \overline{B} + \overline{C})(C + \overline{D} + E)(B + C + D)$

$(A + \overline{B})(A + \overline{B} + C)(A + C)$

A POS expression can contain a single-variable term, as in

$A(A + B + C)(B + C + D)$.

In a POS expression, a single overbar cannot extend over more than one variable; however, more than one variable in a term can have an overbar. For example, a POS expression can have the term $\overline{A} + \overline{B} + \overline{C}$ but not $\overline{A + B + C}$.

Implementation of a POS Expression simply requires ANDing the outputs of two or more OR gates. A sum term is produced by an OR operation and the product of two or more sum terms is produced by an AND operation. Fig.(4-

20) shows for the expression (A + B)(B + C + D)(A + C). The output X of the AND gate equals the POS expression.

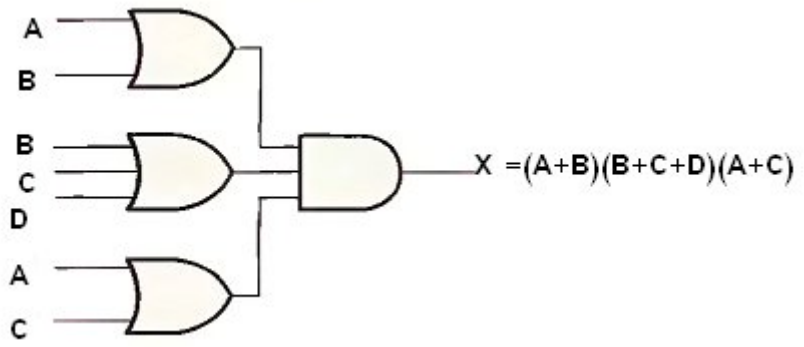


Fig.(4-20)

The Standard POS Form

So far, you have seen POS expressions in which some of the sum terms do not contain all of the variables in the domain of the expression. For example, the expression

$$(A + \overline{B} + C)\,(A + B + \overline{D})\,(A + \overline{B} + \overline{C} + D)$$

has a domain made up of the variables A, B, C, and D. Notice that the complete set of variables in the domain is not represented in e first two terms of the expression; that is, D or $\overline{D}$ is missing from the first term and C or $\overline{C}$ is missing from the second term.

A standard POS expression is one in which all the variables in the domain appear in each sum term in the expression. For example,

$(\overline{A} + \overline{B} + C + D)(A + \overline{B} + C + D)(A + B + C + D)$

is a standard POS expression. Any nonstandard POS expression (referred to simply as POS) can be converted to the standard form using Boolean algebra.

Converting a Sum Term to Standard POS

Each sum term in a POS expression that does not contain all the variables in the domain can be expanded to standard form to include all variables in the domain and their complements. As stated in the following steps, a

nonstandard POS expression is converted into standard form using Boolean algebra rule 8 ($A\,\overline{A} = 0$) from Table 4-1:

Step 1. Add to each nonstandard product term a term made up of the product of the missing variable and its complement. This results in two sum terms. As you know, you can add 0 to anything without changing its value.

Step 2. Apply rule 12 from Table 4-1: $A + BC = (A + B)(A + C)$

Step 3. Repeat Step 1 until all resulting sum terms contain all variables in the domain in either complemented or noncomplemented form.

Example

Convert the following Boolean expression into standard POS form:

$(\overline{A} + B + C)(\overline{B} + C + \overline{D})(A + \overline{B} + \overline{C} + D)$

Solution

The domain of this POS expression is A, B, C, D. Take one term at a time. The first term, $\overline{A} + B + C$, is missing variable D or $\overline{D}$, so add $D\overline{D}$ and apply rule 12 as follows:

$\overline{A} + B + C = \overline{A} + B + C + D\overline{D} = (\overline{A} + B + C + D)(\overline{A} + B + C + \overline{D})$

The second term, $\overline{B} + C + \overline{D}$, is missing variable A or $\overline{A}$, so add $A\overline{A}$ and apply rule 12 as follows:

$\overline{B} + C + \overline{D} = \overline{B} + C + \overline{D} + A\overline{A} = (A + \overline{B} + C + \overline{D})(\overline{A} + \overline{B} + C + \overline{D})$

The third term, $A + B + \overline{C} + \overline{D}$, is already in standard form. The standard POS form of the original expression is as follows:

$(\overline{A} + B + C)(\overline{B} + C + \overline{D})(A + \overline{B} + \overline{C} + D) = (\overline{A} + B + C + D)(\overline{A} + B + C + \overline{D})\,(A + \overline{B} + C + \overline{D})(\overline{A} + \overline{B} + C + \overline{D})\,(A + B + \overline{C} + \overline{D})$

Examples:-

1. Identify each of the following expressions as SOP, standard SOP, POS, or standard POS:
   (a) $AB + \overline{A}BD + \overline{A}C\overline{D}$ (b) $(A + \overline{B} + C)(A + B + \overline{C})$
   (c) $\overline{A}BC + AB\overline{C}$ (d) $A(A + \overline{C})(A + B)$
2. Convert each SOP expression in Question 1 to standard form.
3. Convert each POS expression in Question 1 to standard form.

CANONICAL FORMS OF BOOLEAN EXPRESSIONS

With one variable x & $\overline{x}$.

With two variables $\overline{x}\,\overline{y}$, $x\,\overline{y}$, $\overline{x}\,y$ and $x\,y$.

With three variables $\overline{x}\,\overline{y}\,\overline{z}$, $\overline{x}\,\overline{y}\,z$, $\overline{x}\,y\,\overline{z}$, $\overline{x}\,y\,z$, $x\,\overline{y}\,\overline{z}$, $x\,\overline{y}\,z$, $x\,y\,\overline{z}$ & $x\,y\,z$.

These eight AND terms are called minterms.

n variables can be combined to form 2^n minterms.

x	y	z	minterm	designation	maxterm	designation
0	0	0	$\overline{x}\,\overline{y}\,\overline{z}$	m_0	$x+y+z$	M_0
0	0	1	$\overline{x}\,\overline{y}\,z$	m_1	$x+y+\overline{z}$	M_1
0	1	0	$\overline{x}\,y\,\overline{z}$	m_2	$x+\overline{y}+z$	M_2
0	1	1	$\overline{x}\,y\,z$	m_3	$x+\overline{y}+\overline{z}$	M_3
1	0	0	$x\,\overline{y}\,\overline{z}$	m_4	$\overline{x}+y+z$	M_4
1	0	1	$x\,\overline{y}\,z$	m_5	$\overline{x}+y+\overline{z}$	M_5
1	1	0	$x\,y\,\overline{z}$	m_6	$\overline{x}+\overline{y}+z$	M_6
1	1	1	$x\,y\,z$	m_7	$\overline{x}+\overline{y}+\overline{z}$	M_7
			(AND terms)		(OR terms)	

Note that each maxterm is the complement of its corresponding minterm and vice versa.

For example the function F

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$F = \bar{x}\,\bar{y}\,z + x\,\bar{y}\,\bar{z} + x\,y\,z$

$F = m_1 + m_4 + m_7$

Any Boolean function can be expressed as a sum of minterms (sum of products **SOP**) or product of maxterms (product of sums **POS**).

$\bar{F} = \bar{x}\,\bar{y}\,\bar{z} + \bar{x}\,y\,\bar{z} + \bar{x}\,y\,z + x\,\bar{y}\,z + x\,y\,\bar{z}$

The complement of $\bar{F} = \bar{\bar{F}} = F$

$F = (x + y + z)\,(x + \bar{y} + z)\,(x + \bar{y} + \bar{z})\,(\bar{x} + y + \bar{z})\,(\bar{x} + \bar{y} + z)$

$F = M_0\,M_2\,M_3\,M_5\,M_6$

Example

Express the Boolean function $F = A + \bar{B}C$ in a sum of minterms (SOP).

Solution

The term A is missing two variables because the domain of F is (A, B, C)

$A = A(B + \bar{B}) = AB + A\bar{B}$ because $B + \bar{B} = 1$

$\overline{B}C$ missing A, so

$\overline{B}C(A + \overline{A}) = A\overline{B}C + \overline{A}\overline{B}C$

$AB(C + \overline{C}) = ABC + AB\overline{C}$

$A\overline{B}(C + \overline{C}) = A\overline{B}C + A\overline{B}\overline{C}$

$F = ABC + AB\overline{C} + \underline{A\overline{B}C} + A\overline{B}\overline{C} + \underline{A\overline{B}C} + \overline{A}\overline{B}C$

Because $A + A = A$

$F = ABC + AB\overline{C} + A\overline{B}C + A\overline{B}\overline{C} + \overline{A}\overline{B}C$

$F = m_7 + m_6 + m_5 + m_4 + m_1$

In short notation

$F(A, B, C) = \sum(1, 4, 5, 6, 7)$

$\overline{F}(A, B, C) = \sum(0, 2, 3)$

The complement of a function expressed as the sum of minterms equal to the sum of minterms missing from the original function.

Truth table for $F = A + \overline{B}C$

	A	**B**	**C**	$\overline{\mathbf{B}}$	$\overline{\mathbf{B}}\mathbf{C}$	**F**
0	0	0	0	1	0	0
1	0	0	1	1	1	1
2	0	1	0	0	0	0
3	0	1	1	0	0	0
4	1	0	0	1	0	1
5	1	0	1	1	1	1
6	1	1	0	0	0	1
7	1	1	1	0	0	1

Example

Express $F = xy + \overline{x}z$ in a product of maxterms form.

Solution

$F = xy + \overline{x}z = (xy + \overline{x})(xy + z) = (x + \overline{x})(y + \overline{x})(x + z)(y + z)$

remember $x + \overline{x} = 1$

$F = (y + \overline{x})(x + z)(y + z)$

$F = (\overline{x} + y + z\overline{z})(x + y\overline{y} + z)(x\overline{x} + y + z)$

$F = (\overline{x} + y + z)(\overline{x} + y + \overline{z})(x + y + z)(x + \overline{y} + z)(x + y + z)(\overline{x} + y + z)$

$F = (\overline{x} + y + z)(\overline{x} + y + \overline{z})(x + y + z)(x + \overline{y} + z)$

$F = M_4\, M_5\, M_0\, M_2$

$F(x, y, z) = \prod(0, 2, 4, 5)$

$\overline{F}(x, y, z) = \prod(1, 3, 6, 7)$

The complement of a function expressed as the product of maxterms equal to the product of maxterms missing from the original function.

To convert from one canonical form to another, interchange the symbols $\sum$, $\prod$ and list those numbers missing from the original form.

$F = M_4\, M_5\, M_0\, M_2 = m_1 + m_3 + m_6 + m_7$

$F(x, y, z) = \prod(0, 2, 4, 5) = \sum(1, 3, 6, 7)$

Example

Develop a truth table for the standard SOP expression $\overline{A}\overline{B}C + A\overline{B}\overline{C} + ABC$.

INPUTS			OUTPUT	
A	B	C	X	PRODUCT TERM
0	0	0	0	
0	0	1	1	$\overline{A}\,\overline{B}C$
0	1	0	0	
0	1	1	0	
1	0	0	1	$A\overline{B}\,\overline{C}$
1	0	1	0	
1	1	0	0	
1	1	1	1	ABC

Converting POS Expressions to Truth Table Format

Reca11 that a POS expression is equal to 0 only if at least one of the sum terms is equal to 0. To construct a truth table from a POS expression, list all the possible combinations of binary values of the variables just as was done for the SOP expression. Next, convert the POS expression to standard form if it is not already. Finally, place a 0 in the output column (X) for each binary value that makes the expression a 0 and place a 1 for all the remaining binary values. This procedure is illustrated in Example below:

Example

Determine the truth table for the following standard POS expression:

$$(A + B + C)(A + \overline{B} + C)(A + \overline{B} + \overline{C})(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + C)$$

Solution

There are three variables in the domain and the eight possible binary values are listed in the left three columns of. The binary values that make the sum terms in the expression equal to 0 are A+ B + C: 000; A + $\overline{B}$ + C: 010: A + $\overline{B}$ + $\overline{C}$: 011; $\overline{A}$ + B + $\overline{C}$: 10l; and $\overline{A}$ + $\overline{B}$ + C: 110. For each of these binary values, place a 0 in the output column as shown in the table. For each of the remaining binary combinations, place a 1 in the output column.

INPUTS			OUTPUT	
A	B	C	X	SUM TERM
0	0	0	0	$(A + B + C)$
0	0	1	1	
0	1	0	0	$(A + \overline{B} + C)$
0	1	1	0	$(A + \overline{B} + \overline{C})$
1	0	0	1	
1	0	1	0	$(\overline{A} + B + \overline{C})$
1	1	0	0	$(\overline{A} + \overline{B} + C)$
1	1	1	1	

5 KARNAUGH MAP MINIMIZATION

A Karnaugh map provides a systematic method for simplifying Boolean expressions and, if properly used, will produce the simplest SOP or POS expression possible, known as the minimum expression. As you have seen, the effectiveness of algebraic simplification depends on your familiarity with all the laws, rules, and theorems of Boolean algebra and on your ability to apply them. The Karnaugh map, on the other hand, provides a "cookbook" method for simplification.

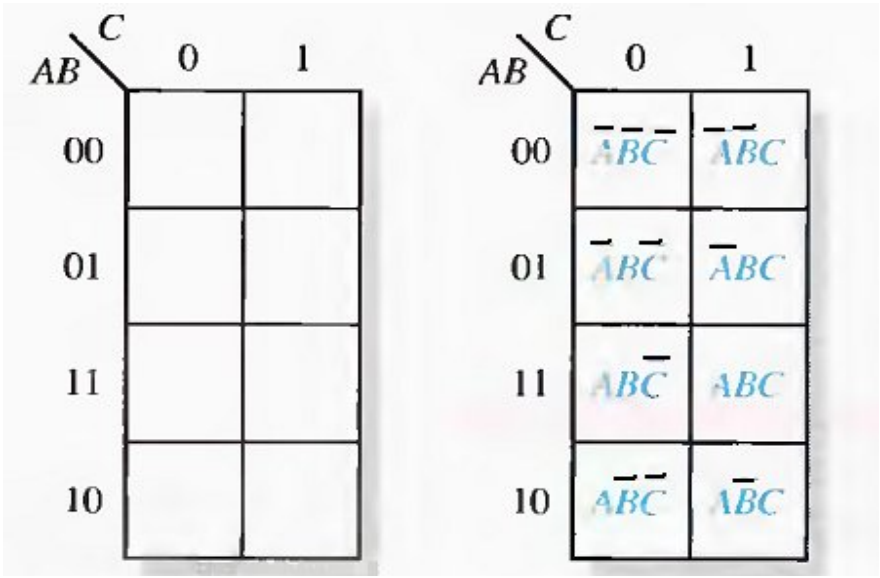
A Karnaugh map is similar to a truth table because it presents all of the possible values of input variables and the resulting output for each value. Instead of being organized into columns and rows like a truth table, the Karnaugh map is an array of cells in which each cell represents a binary value of the input variables. The cells are arranged in a way so that simplification of a given expression is simply a matter of properly grouping the cells. Karnaugh maps can be used for expressions with two, three, four. and five variables. Another method, called the Quine-McClusky method can be used for higher numbers of variables.

The number of cells in a Karnaugh map is equal to the total number of possible input variable combinations as is the number of rows in a truth table. For three variables, the number of cells is $2^3 = 8$. For four variables, the number of cells is $2^4 = 16$.

The 3-Variable Karnaugh Map

The 3-variable Karnaugh map is an array of eight cells. as shown in Fig.(5-1)(a). In this case, A, B, and C are used for the variables although other letters could be used. Binary values of A and B are along the left side (notice

the sequence) and the values of C are across the top. The value of a given cell is the binary values of A and B at the left in the same row combined with the value of C at the top in the same column. For example, the cell in the upper left corner has a binary value of 000 and the cell in the lower right corner has a binary value of 101. Fig.(5-1)(b) shows the standard product terms that are represented by each cell in the Karnaugh map.



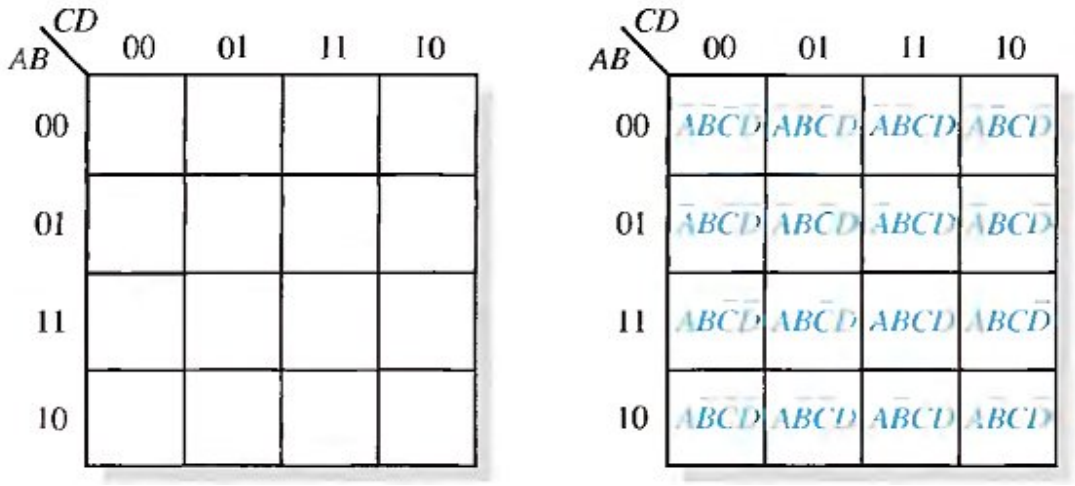
(a) (b)

Fig.(5-1) A 3-variable Karnaugh map showing product terms.

<u>The 4-Variable Karnaugh Map</u>

The 4-variable Karnaugh map is an array of sixteen cells, as shown in Fig.(5-2)(a). Binary values of A and B are along the left side and the values of C and D are across the top. The value of a given cell is the binary values of A and B at the left in the same row combined with the binary values of C and D at the top in the same column. For example, the cell in the upper right corner has a binary value of 0010 and the cell in the lower right corner has a

binary value of 1010. Fig.(5-2)(b) shows the standard product terms that are represented by each cell in the 4-variable Karnaugh map.



(a) (b)

Fig.(5-2) A 4-variable Karnaugh map.

Cell Adjacency

The cells in a Karnaugh map are arranged so that there is only a single-variable change between adjacent cells. Adjacency is defined by a single-variable change. In the 3-variable map the 010 cell is adjacent to the 000 cell, the 011 cell, and the 110 cell. The 010 cell is not adjacent to the 001 cell, the 111 cell, the 100 cell, or the 101 cell.

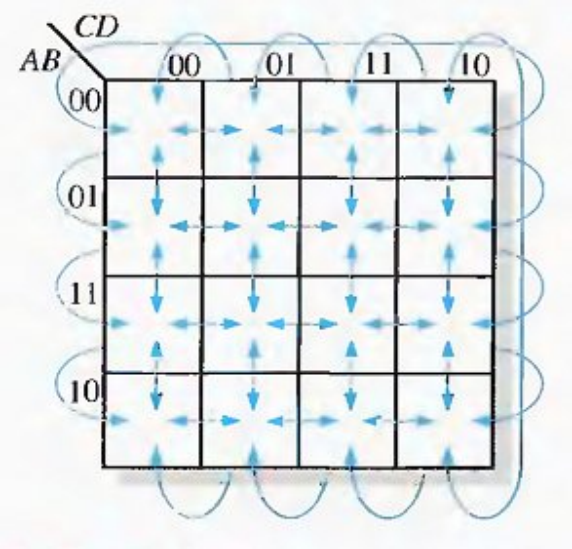


Fig.(5-3) Adjacent cells on a Karnaugh map are those that differ by only one variable. Arrows point between adjacent cells.

KARNAUGH MAP SOP MINIMIZATION

For an SOP expression in standard form, a 1 is placed on the Karnaugh map for each product term in the expression. Each 1 is placed in a cell corresponding to the value of a product term. For example, for the product term ABC, a 1 goes in the 10l cell on a 3-variable map.

Example

Map the following standard SOP expression on a Karnaugh map:

see Fig.(5-4).

Example

Map the following standard SOP expression on a Karnaugh map:

$$\bar{A}\bar{B}CD + \bar{A}B\bar{C}\bar{D} + AB\bar{C}D + ABCD + AB\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + A\bar{B}C\bar{D}$$

See Fig.(5-5).

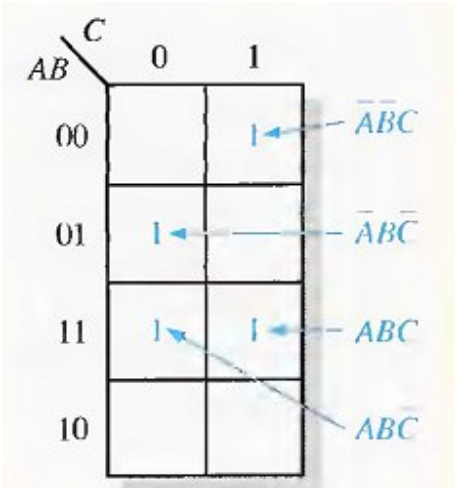


Fig.(5-4)

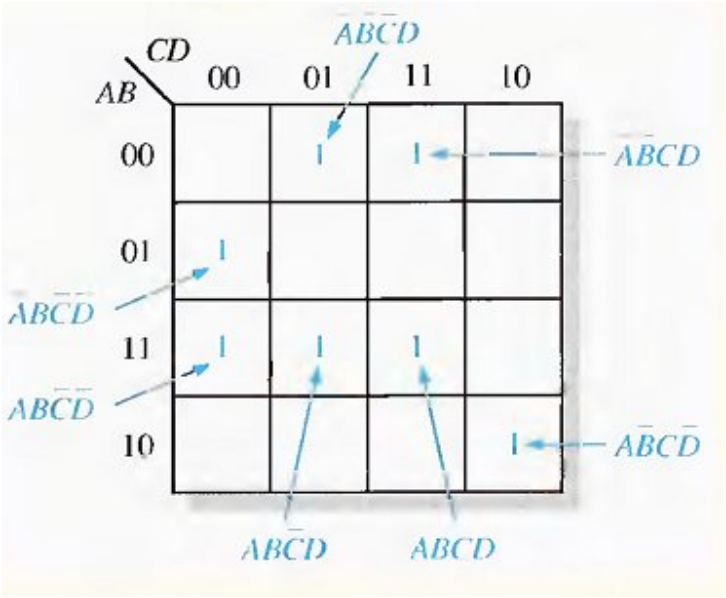


Fig.(5-5)

Example

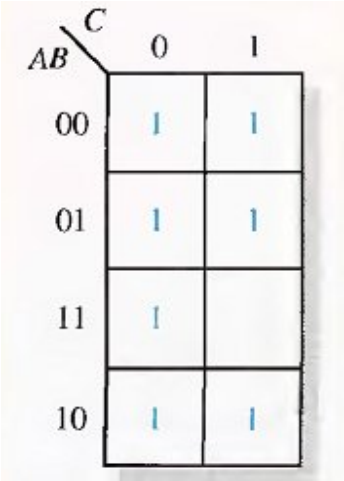
Map the following SOP expression on a Karnaugh map: $\bar{A} + A\bar{B} + AB\bar{C}.$

Solution

The SOP expression is obviously not in standard form because each product term does not have three variables. The first term is missing two variables,

the second term is missing one variable, and the third term is standard. First expand the terms numerically as follows:

$\overline{A}$	$+ A\overline{B}$	$+ AB\overline{C}$
000	100	110
001	101	
010		
011		



Example

Map the following SOP expression on a Karnaugh map:

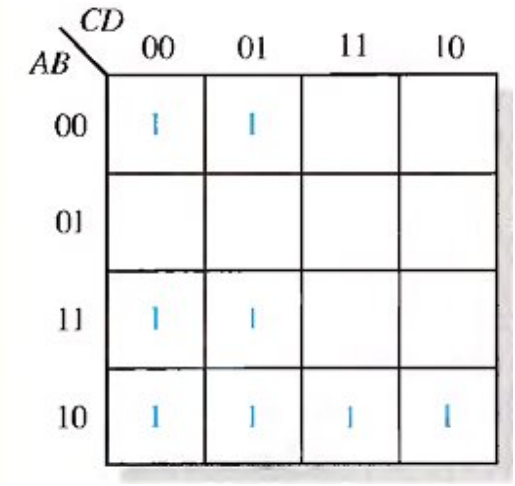
$$\overline{B}\,\overline{C} + A\overline{B} + AB\overline{C} + A\overline{B}C\overline{D} + \overline{A}\,\overline{B}\,\overline{C}D + A\overline{B}CD$$

Solution

The SOP expression is obviously not in standard form because each product term does not have four variables.

$\overline{B}\,\overline{C}$	$A\overline{B}$	$+$	$AB\overline{C}$	$+ A\overline{B}C\overline{D}$	$+ \overline{A}\,\overline{B}\,\overline{C}D$	$+ A\overline{B}CD$
0000	1000		1100	1010	0001	1011
0001	1001		1101			
1000	1010					
1001	1011					

Map each of the resulting binary values by placing a 1 in the appropriate cell of the 4- variable Karnaugh map.



Karnaugh Map Simplification of SOP Expressions

Grouping the 1s, you can group 1s on the Karnaugh map according to the following rules by enclosing those adjacent cells containing 1s. The goal is to maximize the size of the

groups and to minimize the number of groups.

- A group must contain either 1, 2, 4, 8, or 16 cells, which are all powers of two. In the case of a 3-variable map, $2^3 = 8$ cells is the maximum group.
- Each cell in a group must be adjacent to one or more cells in that same group.
- Always include the largest possible number of 1s in a group in accordance with rule 1.
- Each 1 on the map must be included in at least one group. The 1s already in a group can be included in another group as long as the overlapping groups include noncommon 1s.

Example:

Group the 1s in each of the Karnaugh maps in Fig.(5-6).

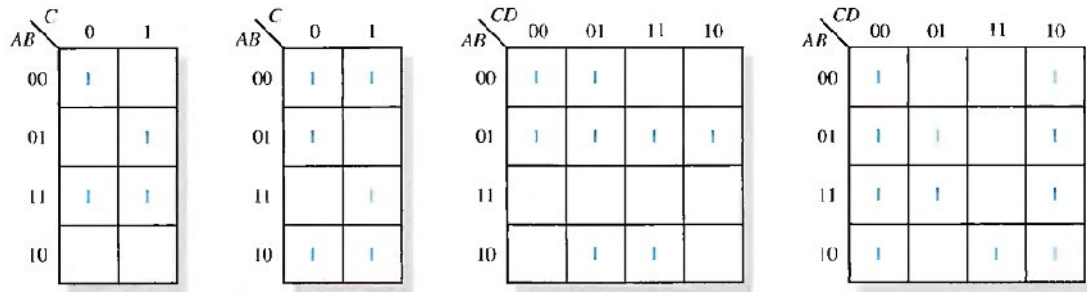


Fig.(5-6)

Solution:

The groupings are shown in Fig.(5-7). In some cases, there may be more than one way to group the 1s to form maximum groupings.

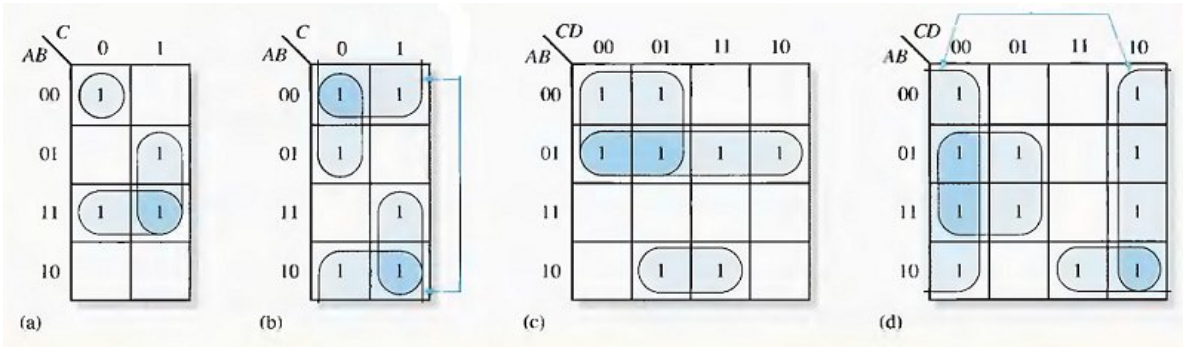


Fig.(5-7)

Determine the minimum product term for each group.

a. For a 3-variable map:

(1) A l-cell group yields a 3-variable product term

(2) A 2-cell group yields a 2-variable product term

(3) A 4-cell group yields a 1-variable term

(4) An 8-cell group yields a value of 1 for the expression

b. For a 4-variable map:

(1) A 1-cell group yields a 4-variable product term

(2) A 2-cell group yields a 3-variable product term

(3) A 4-cell group yields a 2-variable product term

(4) An 8-cell group yields a 1-variable term

(5) A 16-cell group yields a value of 1 for the expression

Example:

Determine the product terms for each of the Karnaugh maps in Fig.(5-7) and write the resulting minimum SOP expression.

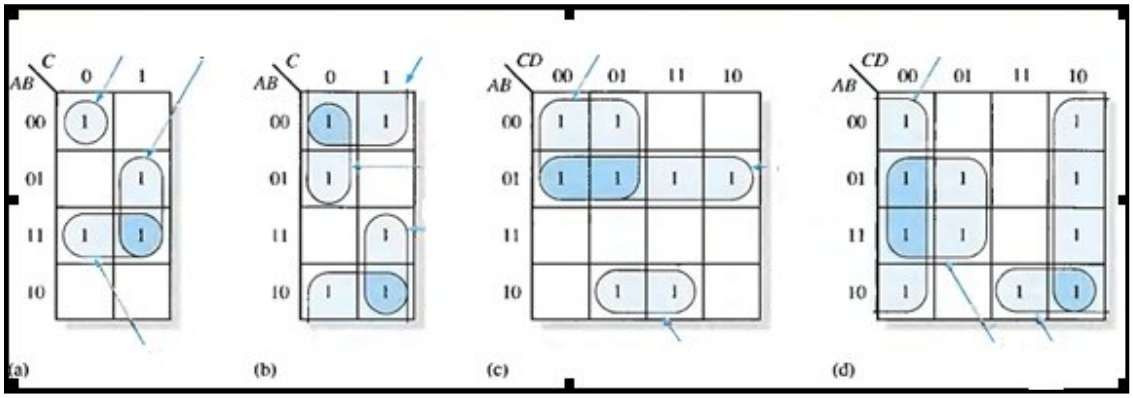


Fig.(5-8)

Solution:

The resulting minimum product term for each group is shown in Fig.(5-8).

The minimum SOP expressions for each of the Karnaugh maps in the figure are:

(a) $AB+BC+\overline{A}\overline{B}\overline{C}$

(b) $\overline{B} + A\,C + \overline{A}\overline{C}$

(C) $\overline{A}B + \overline{A}\overline{C} + A\overline{B}D$

(d) $\overline{D} + A\overline{B}C + B\overline{C}$

Example: Use a Karnaugh map to minimize the following standard SOP expression:

$$\overline{A}\,\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\,\overline{C} + A\overline{B}C + \overline{A}BC$$

Example: Use a Karnaugh map to minimize the following SOP expression:

$$\overline{B}\,\overline{C}\,\overline{D} + \overline{A}B\overline{C}\,\overline{D} + AB\overline{C}\,\overline{D} + \overline{A}\,\overline{B}CD + A\overline{B}CD + \overline{A}\,\overline{B}C\overline{D} + \overline{A}BC\overline{D} + ABC\overline{D} + A\overline{B}C\overline{D}$$

"Don't Care" Conditions

Sometimes a situation arises in which some input variable combinations are not allowed. For example, recall that in the BCD code there are six invalid combinations: 1010, 1011, 1100, 1101, 1110, and 1111. Since these unallowed states will never occur in an application involving the BCD code, they can be treated as "don't care" terms with respect to their effect on the output. That is, for these "don't care" terms either a 1 or a 0 may be assigned to the output: it really does not matter since they will never occur.

The "don't care" terms can be used to advantage on the Karnaugh map. Fig.(5-9) shows that for each "don't care" term, an X is placed in the cell. When grouping the 1 s, the Xs can be treated as 1s to make a larger grouping or as 0s if they cannot be used to advantage. The larger a group, the simpler the resulting term will be.

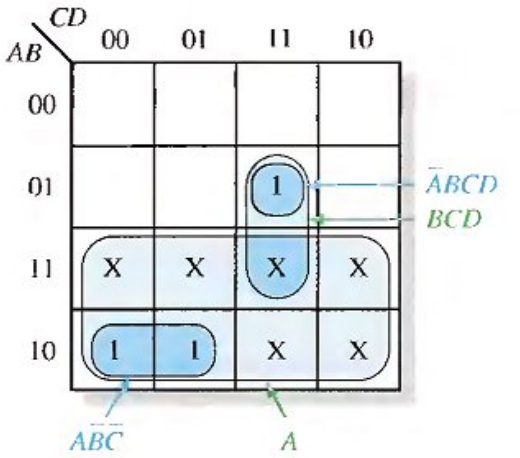
The truth table in Fig.(5-9)(a) describes a logic function that has a 1 output only when the BCD code for 7,8, or 9 is present on the inputs. If the "don't cares" are used as 1s, the resulting expression for the function is A + BCD, as indicated in part (b). If the "don't cares" are not used as 1s, the resulting

expression is ABC + ABCD: so you can see the advantage of using "don't care" terms to get the simplest expression.

Inputs A B C D	Output Y
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	0
0 1 0 0	0
0 1 0 1	0
0 1 1 0	0
0 1 1 1	1
1 0 0 0	1
1 0 0 1	1
1 0 1 0	X
1 0 1 1	X
1 1 0 0	X
1 1 0 1	X
1 1 1 0	X
1 1 1 1	X

(a) Truth table

Don't cares



(b) Without "don't cares" $Y = A\bar{B}\bar{C} + \bar{A}BCD$
With "don't cares" $Y = A + BCD$

Fig.(5-9)

KARNAUGH MAP POS MINIMIZATION

In this section, we will focus on POS expressions. The approaches are much the same except that with POS expressions, 0s representing the standard sum terms are placed on the Karnaugh map instead of 1s.

For a POS expression in standard form, a 0 is placed on the Karnaugh map for each sum term in the expression. Each 0 is placed in a cell corresponding to the value of a sum term. For example, for the sum term $A + \overline{B} + C$, a 0 goes in the 0 1 0 cell on a 3-variable map.

When a POS expression is completely mapped, there will be a number of 0s on the Karnaugh map equal to the number of sum terms in the standard POS expression. The cells that do not have a 0 are the cells for which the expression is 1. Usually, when working with POS expressions, the 1s are left off. The following steps and the illustration in Fig.(5-10) show the mapping process.

Step 1. Determine the binary value of each sum term in the standard POS expression. This is the binary value that makes the term equal to 0.

Step 2. As each sum term is evaluated, place a 0 on the Karnaugh map in the corresponding cell.

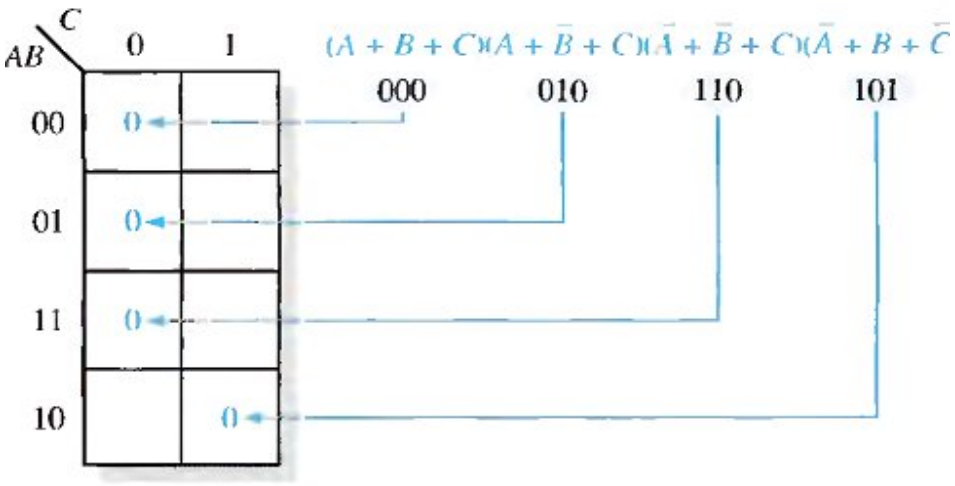


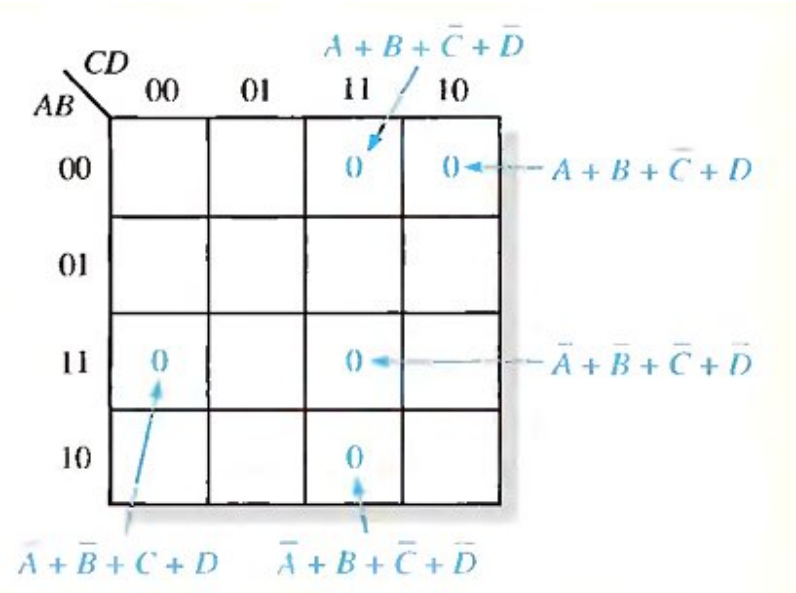
Fig.(5-10)
Example of mapping a standard POS expression.

Example:
Map the following standard POS expression on a Karnaugh map:

$$(\bar{A} + \bar{B} + C + D)(\bar{A} + B + \bar{C} + \bar{D})(A + B + \bar{C} + D)(\bar{A} + \bar{B} + \bar{C} + \bar{D})(A + B + \bar{C} + \bar{D})$$

Solution:

$$(\bar{A} + \bar{B} + C + D)(\bar{A} + B + \bar{C} + \bar{D})(A + B + \bar{C} + D)(\bar{A} + \bar{B} + \bar{C} + \bar{D})(A + B + \bar{C} + \bar{D})$$
$$1100 \qquad 1011 \qquad 0010 \qquad 1111 \qquad 0011$$



Karnaugh Map Simplification of POS Expressions

The process for minimizing a POS expression is basically the same as for an SOP expression except that you group 0s to produce minimum sum terms instead of grouping 1s to produce minimum product terms. The rules for grouping the 0s are the same as those for grouping the 1s that you learned before.

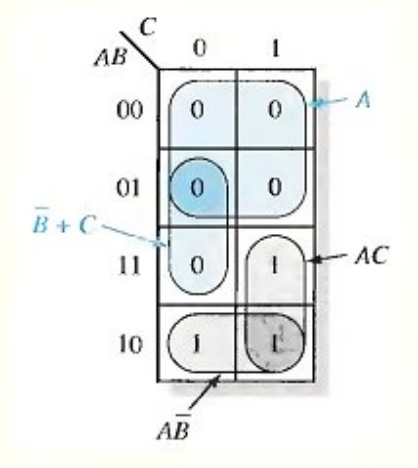
Example:

Use a Karnaugh map to minimize the following standard POS expression:

Also, derive the equivalent SOP expression.

$$(A + B + C)(A + B + \overline{C})(A + \overline{B} + C)(A + \overline{B} + \overline{C})(\overline{A} + \overline{B} + C)$$

Solution:



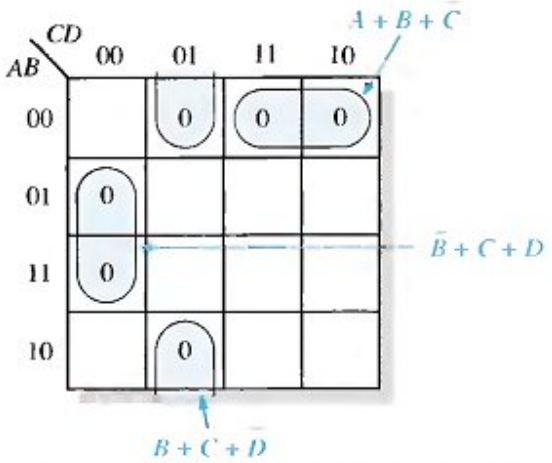
Example: Use a Karnaugh map to minimize the following POS expression:

$$(B + C + D)(A + B + \overline{C} + D)(\overline{A} + B + C + \overline{D})(A + \overline{B} + C + D)(\overline{A} + \overline{B} + C + D)$$

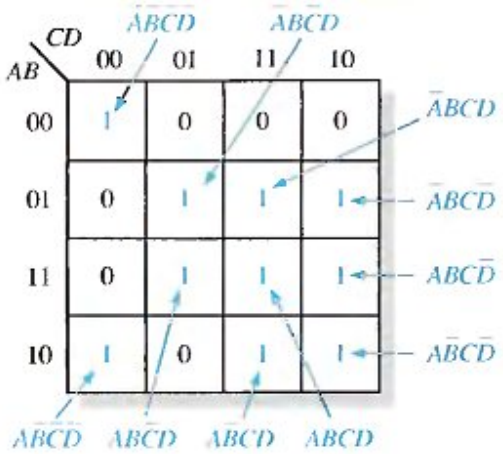
Example: Using a Karnaugh map, convert the following standard POS expression into a minimum POS expression, a standard SOP expression, and

a minimum SOP expression.

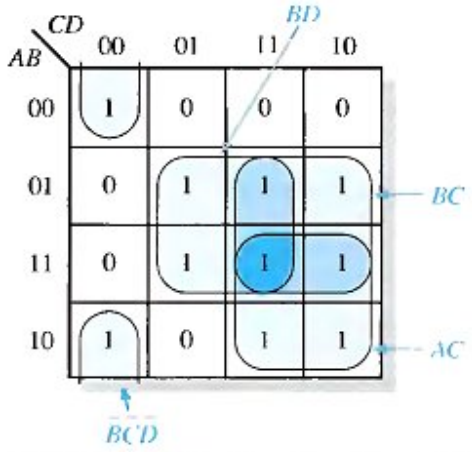
$$(\overline{A} + \overline{B} + C + D)(A + \overline{B} + C + D)(A + B + C + \overline{D})$$
$$(A + B + \overline{C} + \overline{D})(\overline{A} + B + C + \overline{D})(A + B + \overline{C} + D)$$



(a) Minimum POS: $(A + B + C)(\overline{B} + \overline{C} + D)(B + C + \overline{D})$



(b) Standard SOP:
$\overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + \overline{A}BCD + \overline{A}BC\overline{D} + ABC\overline{D} + A\overline{B}C\overline{D} +$
$A\overline{B}\overline{C}\overline{D} + AB\overline{C}\overline{D} + A\overline{B}CD + ABCD$



(c) Minimum SOP: $AC + BC + BD + \bar{B}\bar{C}\bar{D}$

FIVE-VARIABLE KARNAUGH MAPS

Boolean functions with five variables can be simplified using a 32-cell Karnaugh map. Actually, two 4-variable maps (16 cells each) are used to construct a 5-variable map. You already know the cell adjacencies within each of the 4- variable maps and how to form groups of cells containing 1s to simplify an SOP expression. All you need to learn for five variables is the cell adjacencies between the two 4-variable maps and how to group those adjacent 1s.

A Karnaugh map for five variables (ABCDE) can be constructed using two 4-variable maps with which you are already familiar. Each map contains 16 cells with all combinations of variables B, C, D, and E. One map is for A = 0 and the other is for A = 1, as shown in Fig.(5-11).

Cell Adjacencies

You already know how to determine adjacent cells within the 4-variable map. The best way to visualize cell adjacencies between the two 16-cel1 maps is to imagine that the A = 0 map is placed on top of the A = 1 map. Each cell in the A = 0 map is adjacent to the cell directly below it in the A = 1 map, see Fig.(5-12).

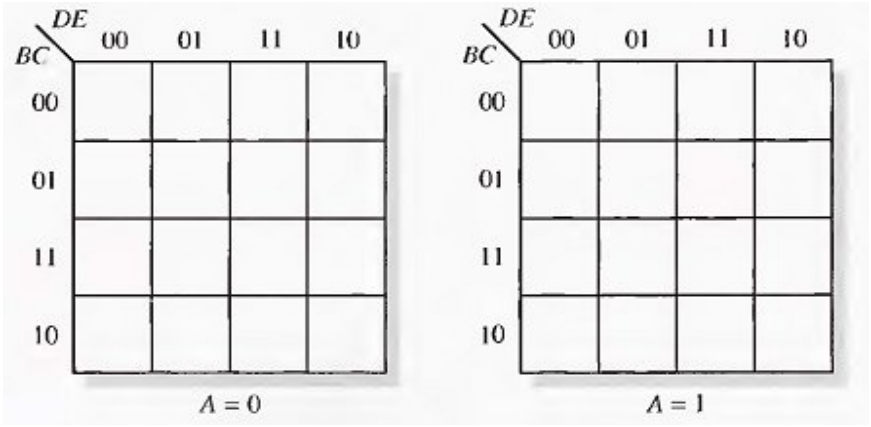


Fig.(5-11)

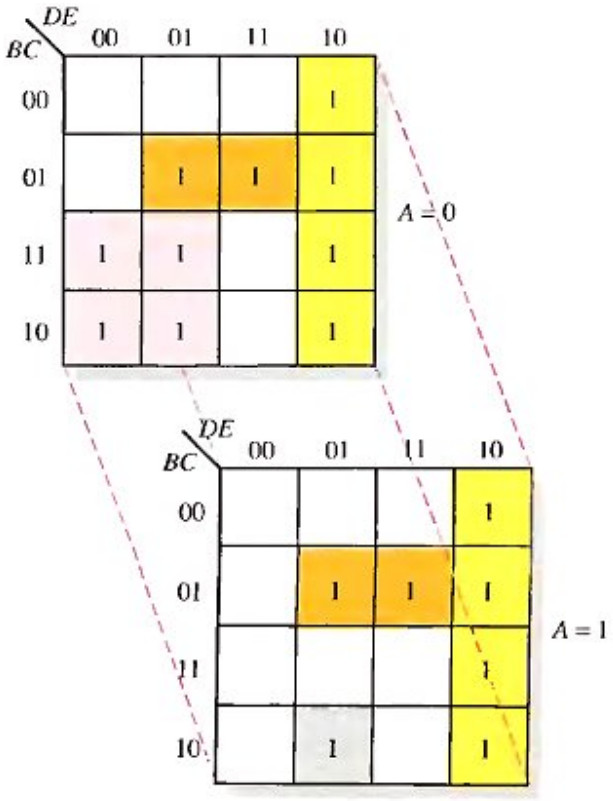


Fig.(5-12)

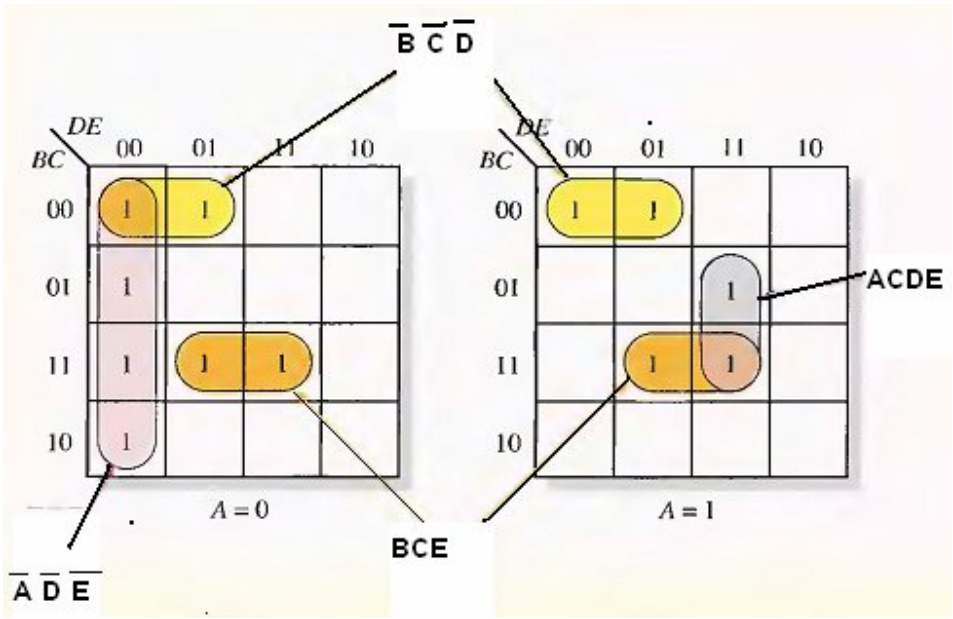
The simplified SOP expression yields

$x = D\overline{E} + \overline{B}CE + \overline{A}B\overline{D} + B\overline{C}\,\overline{D}E$

Example:

Use a Karnaugh map to minimize the following standard SOP 5-variable expression:

$$X = \bar{A}\bar{B}\bar{C}\bar{D}\bar{E} + \bar{A}\bar{B}C\bar{D}\bar{E} + \bar{A}BC\bar{D}\bar{E} + \bar{A}BC\bar{D}E + \bar{A}\bar{B}\bar{C}\bar{D}E + \bar{A}BC\bar{D}E + \bar{A}BCDE + A\bar{B}\bar{C}\bar{D}\bar{E} + A\bar{B}\bar{C}\bar{D}E + ABC\bar{D}E + ABCDE + A\bar{B}CDE$$



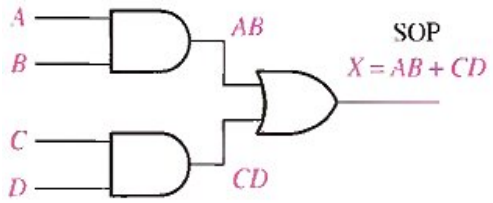
$$X = \bar{A}\bar{D}\bar{E} + \bar{B}\bar{C}\bar{D} + BCE + ACDE$$

6 COMBINATIONAL LOGIC ANALISIS

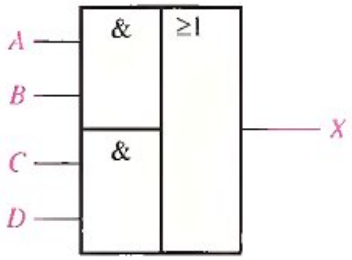
1- AND-OR Logic

Fig.(6-1)(a) shows an AND-OR circuit consisting of two 2-input AND gates and one 2-input OR gate; Fig.(6-1)(b) is the ANSI standard rectangular outline symbol. The Boolean expressions for the AND gate outputs and the resulting SOP expression for the output X are shown in the diagram. In general, all AND-OR circuit can have any number of AND gates each with any number of inputs.

The truth table for a 4-input AND-OR logic circuit is shown in Table 6-1. The intermediate AND gate outputs (AB and CD columns) are also shown in the table.



(a) Logic diagram



(b) ANSI standard rectangular outline symbol.

Fig.(6-1)

For a 4-input AND-OR logic circuit, the output X is HIGH (1) if both input A and input B are HIGH (1) or both input C and input D are HIGH (1).

2-AND-OR-Invert Logic

When the output of an AND-OR circuit is complemented (inverted), it results in an AND-OR-Invert circuit. Recall that AND-OR logic directly implements SOP expressions. POS expressions can be implemented with AND-OR-Invert logic. This is illustrated as follows, starting with a POS expression and developing the corresponding AND-OR-Invert expression.

Table 6-1

INPUTS						OUTPUT
A	B	C	D	AB	CD	X
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	1	1
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	0	1	1
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	0	0	0
1	0	1	1	0	1	1
1	1	0	0	1	0	1
1	1	0	1	1	0	1
1	1	1	0	1	0	1
1	1	1	1	1	1	1

$$X = (\overline{A} + \overline{B})(\overline{C} + \overline{D}) = (\overline{AB})(\overline{CD}) = \overline{\overline{(\overline{AB})(\overline{CD})}} = \overline{\overline{AB} + \overline{CD}} = \overline{AB + CD}$$

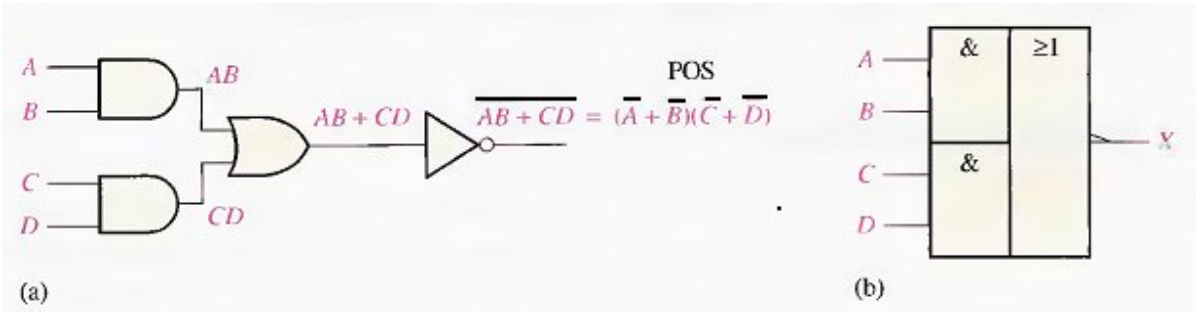


Fig.(6-2)

For a 4-input AND-OR-Invert logic circuit, the output X is LOW (0) if both input A and input B are HIGH (1) or both input C and input D are HIGH (1).

3-Exclusive-OR logic

The exclusive-OR gate was introduced before. Although, because of its importance, this circuit is considered a type of logic gate with its own unique symbol it is actually a combination of two AND gates, one OR gate, and two inverters, as shown in Fig.(6-3)(a). The two standard logic symbols are shown in parts (b) and (c).

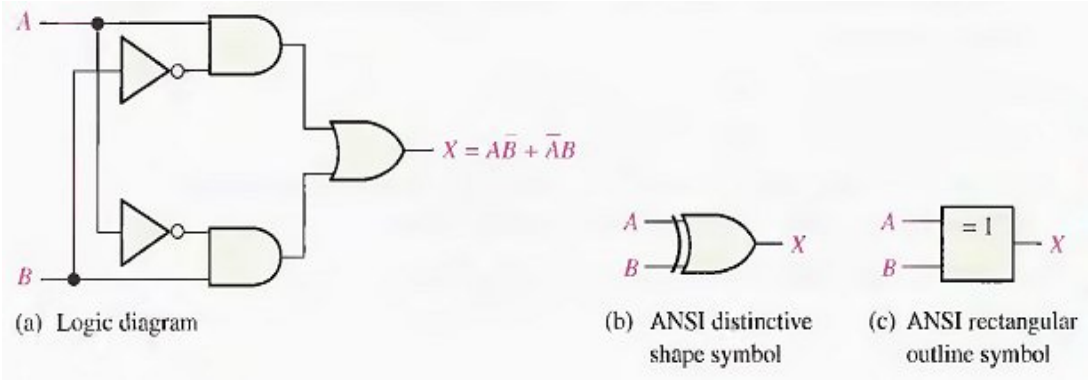


Fig.(6-3)

The output expression for the circuit in Fig.(6-3) is

$$X = A\bar{B} + \bar{A}B$$

Can be written as $X = A \oplus B$

Table 6-2 Truth table for an exclusive-OR.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

4- Exclusive-NOR Logic

As you know, the complement of the exclusive-OR function is the exclusive-NOR, whichis derived as follows:

$$X = \overline{A\overline{B} + \overline{A}B} = \overline{(A\overline{B})}\ \overline{(\overline{A}B)} = (\overline{A} + B)(A + \overline{B}) = \overline{A}\,\overline{B} + AB$$

Notice that the output X is HIGH only when the two inputs, A and B, are at the same level.

The exclusive-NOR can be implemented by simply inverting the output of an exclusive- OR, as shown in Fig(6-4)(a), or by directly implementing the expression $\overline{A}\,\overline{B} + AB$, as shown in part (b).

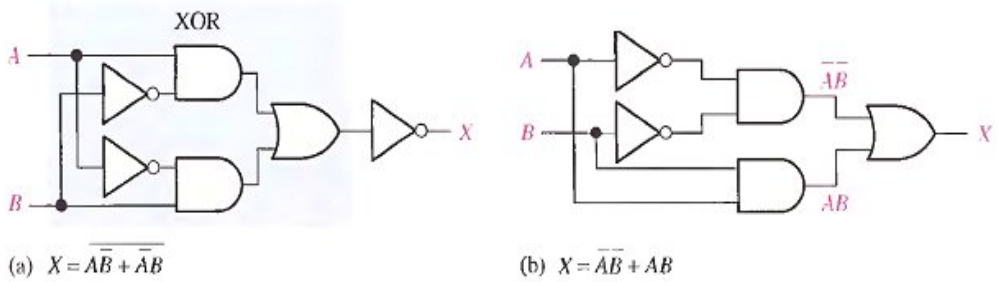


Fig.(6-4)

Example

Develop a logic circuit with four input variables that will only produce a 1 output when exactly three input variables are 1s. Fig.(6-5) shows the circuit.

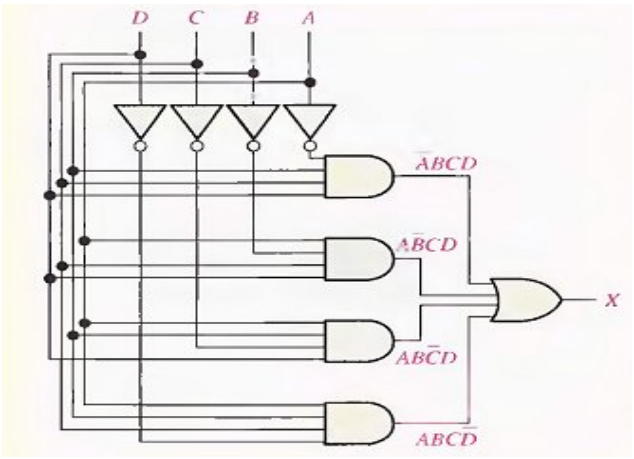


Fig.(6-5)

Example

Reduce the combinational logic circuit in Fig.(6-6) to a minimum form.

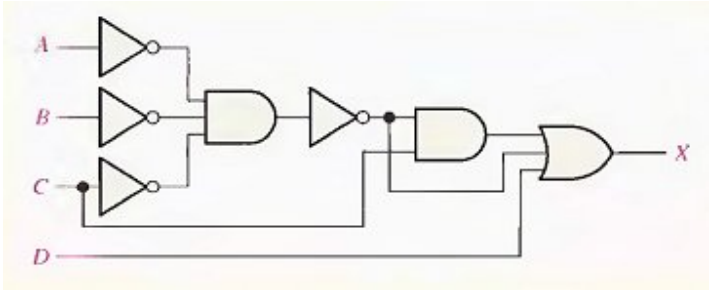


Fig.(6-6)

Solution

The expression for the output of the circuit is

$X = (\overline{A\,B\,C})\,C + \overline{A}\,\overline{B}\,\overline{C} + D$

Applying DeMorgan's theorem and Boolean algebra,

$$\begin{aligned} X &= (\overline{\overline{A} + \overline{B} + \overline{C}})C + \overline{\overline{A} + \overline{B} + \overline{C}} + D \\ &= AC + BC + CC + A + B + C + D \\ &= AC + BC + C + A + B + C + D \\ &= C(A + B + 1) + A + B + D \\ X &= A + B + C + D \end{aligned}$$

The simplified circuit is a 4-input OR gate as shown in Fig.(6-7).

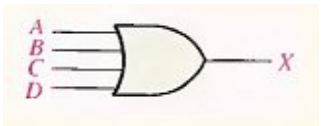


Fig.(6-7)

THE UNIVERSAL PROPERTY OF NAND AND NOR GATES

1- The NAND Gate as a Universal Logic Element

The NAND gate is a universal gate because it can be used to produce the NOT, the AND, the OR, and the NOR functions. An inverter can be made from a NAND gate by connecting all of the inputs together and creating, in effect, a single input, as shown in Fig.(6-8)(a) for a 2-input gate. An AND function can be generated by the use of NAND gates alone, as shown in Fig.(6-8)(b). An OR function can be produced with only NAND gates, as illustrated in part (c). Finally. a NOR function is produced as shown in part (d).

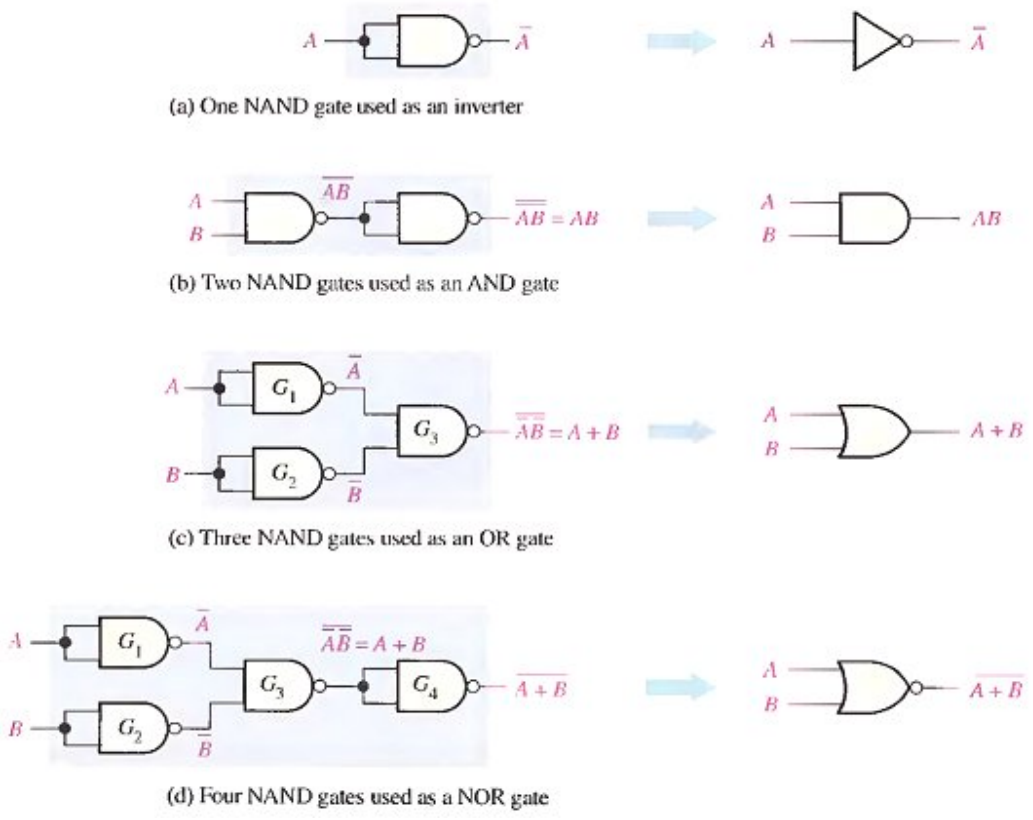


Fig.(6-9)

2- The NOR Gate as a Universal Logic Element

Like the NAND gate, the NOR gate can be used to produce the NOT, AND. OR and NAND functions. A NOT circuit, or inverter, can be made from a NOR gate by connecting all of the inputs together to effectively create a single input, as shown in Fig.(6-10)(a) with a 2-input example. Also, an OR gate can be produced from NOR gates, as illustrated in Fig.(6-10)(b). An AND gate can be constructed by the use of NOR gates, as shown in Fig.(6-10)(c). In this case the NOR gates G 1 and G 2 are used as inverters, and the final output is derived by the use of DeMorgan's theorem as follows:

$$X=\overline{\overline{A}+\overline{B}}=AB$$

Fig.(6-10)(d) shows how NOR gates are used t0 form a NAND function.

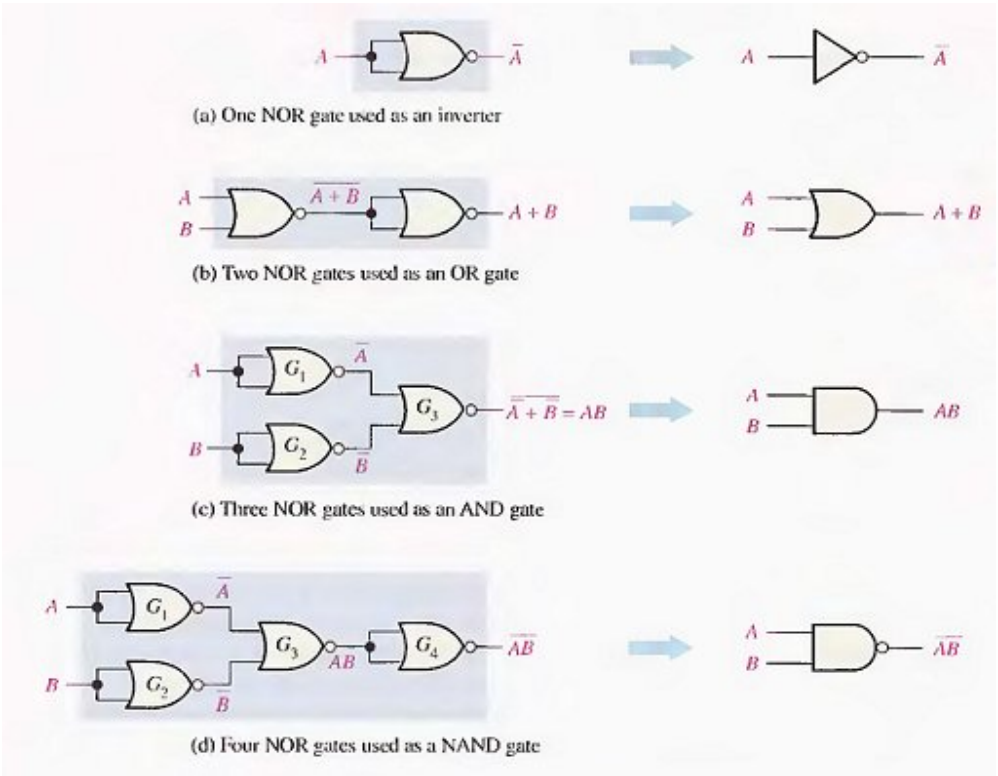


Fig.(6-10)

Example

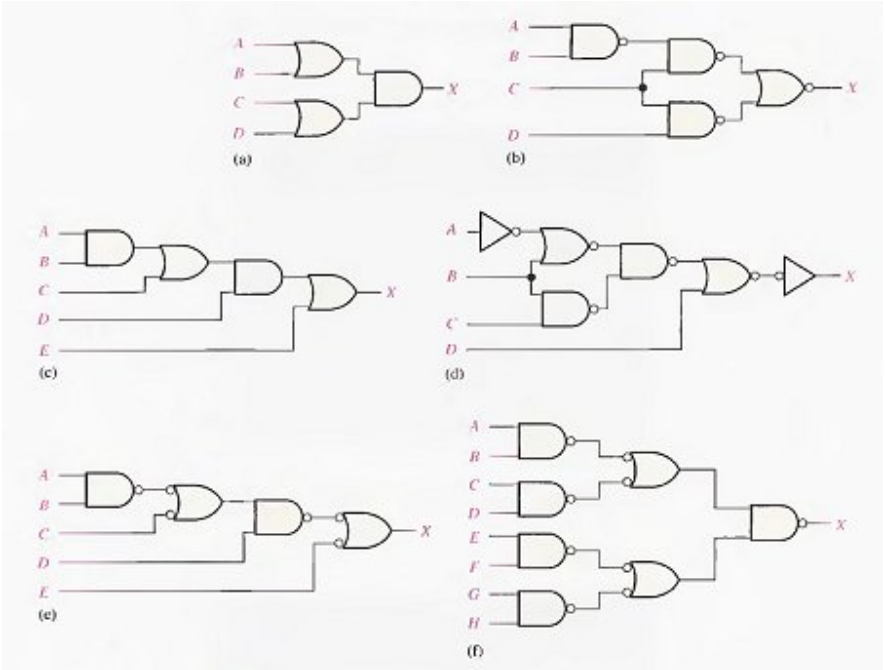
1. Use NAND gates to implement each expression:
   (a) $X = \overline{A} + B$ (b) $X = A\overline{B}$
2. Use NOR gates to implement each expression:
   (a) $X = \overline{A} + B$ (b) $X = A\overline{B}$

Example

1- **Write the output expression for each circuit as it appears in Fig.(6-11) and then change each circuit to an equivalent AND-OR configuration.**

2- **Develop the truth table for circuit in Fig.(6-11)(a-b).**

3- **Show that an exclusive-NOR circuit produces a POS output.**



7 FUNCTIONS

OF COMBINATIONAL LOGIC

7-1 BASIC ADDERS

A half-adder adds two bits and produces a sum and a carry output. Adders are important in computers and also in other types of digital systems in which numerical data are processed.

The Half-Adder

Recall the basic rules for binary.

0+ 0= 0

0+ 1= 1

1+ 0= 1

1 + 1 = 10

The operations are performed by a logic circuit called a half-adder.

The half-adder accepts two binary digits on its inputs and produces two binary

digits on its outputs, a sum bit and a carry bit.

A half-adder is represented by the logic symbol in Fig.(7-1).

Half-Adder Logic: From the operation of the half-adder as stated in Table 7-1, expressions can be derived for the sum and the output carry as functions of the inputs. Notice that the output Carry (C_{out}) is a 1 only when both A and B are 1s: therefore. C_{out} can be expressed

$C_{out} = AB$

Now observe that the sum output ($\sum$) is a 1 only if the input variables A and B, are not equal. The sum can therefore be expressed as the exclusive-OR of the input variables.

$\sum = A \oplus B$

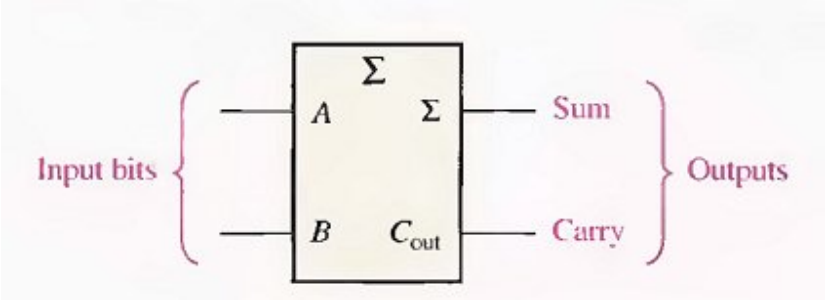


Fig.(7-1) Logic symbol for a half-adder.

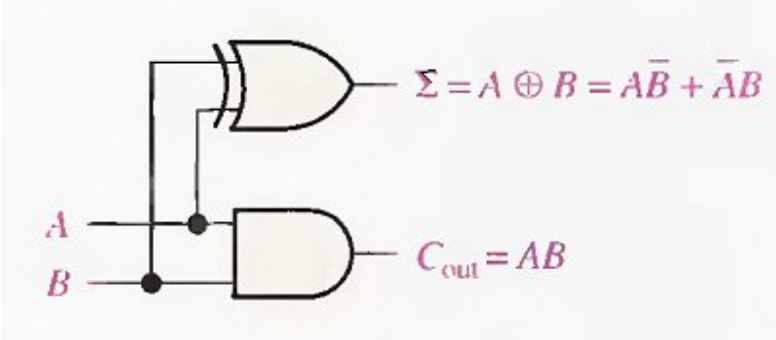


Fig.(7-2) Half-adder logic diagram.

Table 7-1

A	B	C_{out}	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

The Full-Adder

The second category of adder is the full-adder. The full-adder accepts two input bits and an input carry and generates a sum output and an output carry.

The basic difference between a full-adder and a half-adder is that the full-adder accepts an input carry. A logic symbol for a full-adder is shown in Fig.(7-3), and the truth table in Table 7-2 shows the operation of a full-adder.

Table 7-2

A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

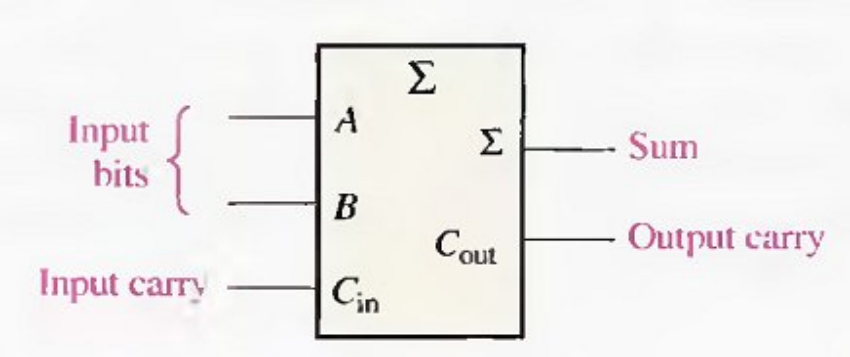


Fig.(7-3) Logic symbol for a full-adder.

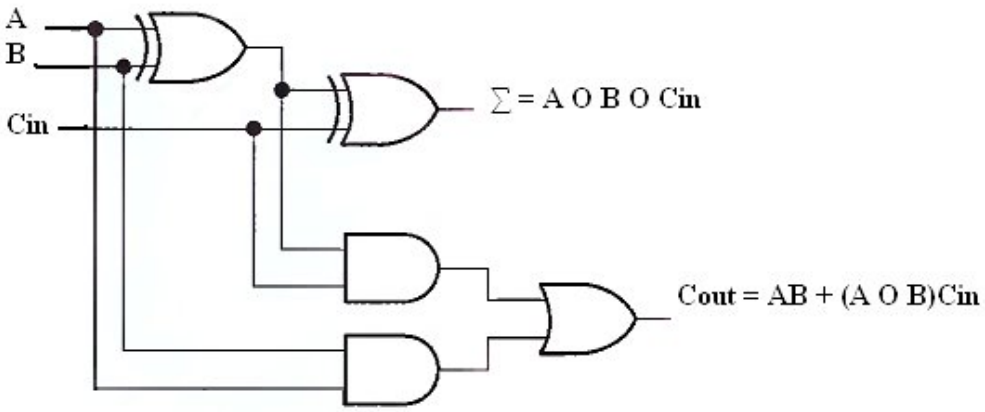


Fig.(7-4) Complete logic circuit for a full-adder.

$$\sum = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + (A \oplus B)C_{in}$$

Notice in Fig.(7-4) there are two half-adders, connected as shown in the block diagram of Fig.(7-5), with their output carries ORed.

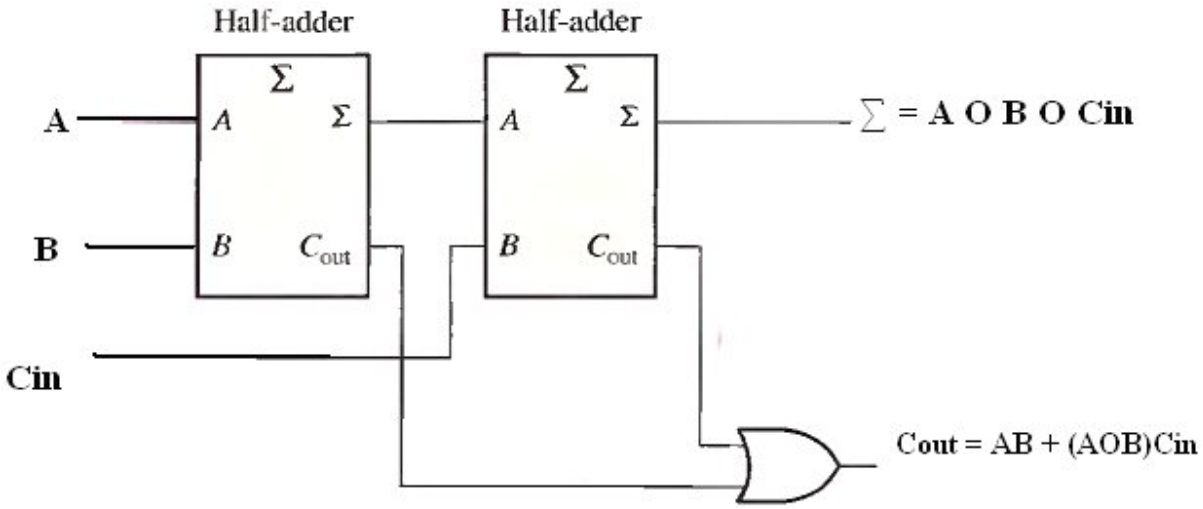


Fig.(7-5) Arrangement of two half-adders to form a full-adder.

Example: For each of the three full-adders in Fig.(7-6), determine the outputs for the inputs shown.

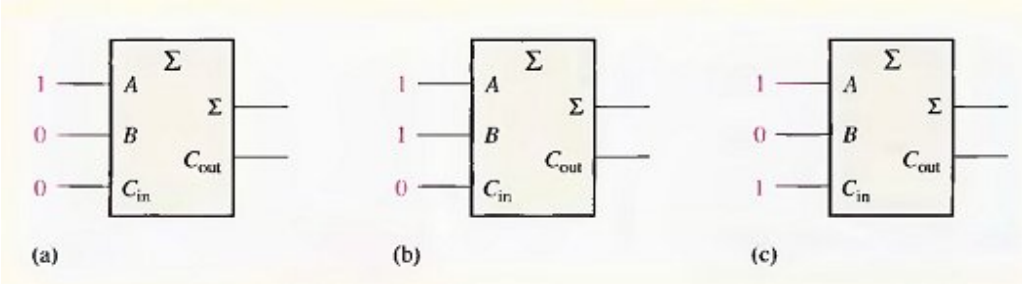
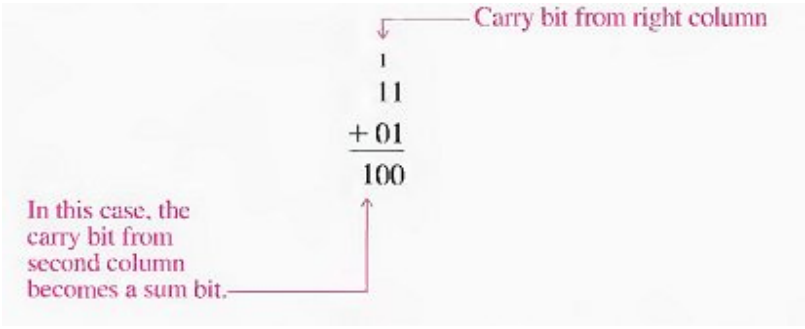


Fig.(7-6)

7-2 PARALLEL BINARY ADDERS

As you saw in Section 7-1, a single full-adder is capable of adding two 1-bit numbers and an input carry. To add binary numbers with more than one bit, you must use additional full-adders. When one binary number is added to another, each column generates a sum bit and a 1 or 0 carry bit to the next column to the left, as illustrated here with 2-bit numbers.



To add two binary numbers, a full-adder is required for each bit in the numbers. So for 2-bit numbers, two adders are needed; for 4-bit numbers, four adders are used; and so on. The carry output of each adder is connected to the carry input of the next higher-order adder, as shown in Fig.(7-7) for a 2-bit adder. Notice that either a half-adder can be used for the least significant position or the carry input of a full-adder can be made 0 (grounded) because there is no carry input to the least significant bit position.

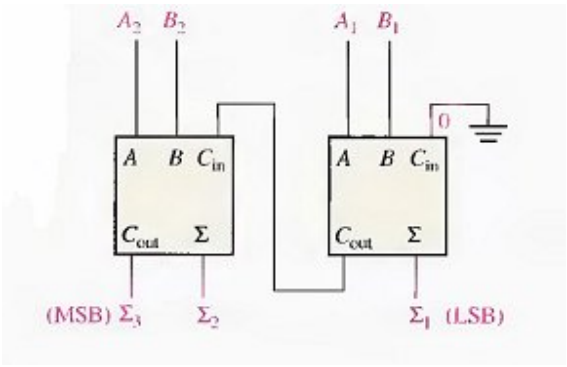


Fig.(7-7)Block diagram of a basic 2-bit parallel adder using two full-adders.

Example: Determine the sum generated by the 3-bit parallel adder in Fig.(7-8) and show the intermediate carries when the binary numbers 101 and 011 are being added.

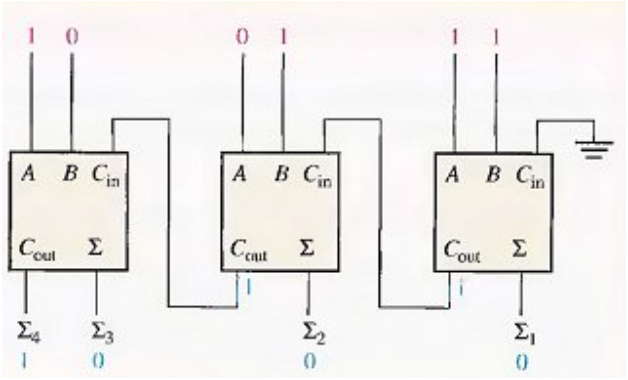


Fig.(7-8)

Four-Bit Parallel Adders

A group of four bits is called a nibble. A basic 4-bit parallel adder is implemented with four full-adder stages as shown in Fig.(7-9). Again, the LSBs (A1 and B1) in each number being added go into the right-most full-adder: the higher-order bits are applied as shown to the successively higher-order added, with the MSBs (A and B) in each number being applied to the left-most full-adder. The Carry output of each adder is connected to the carry input of the next higher-order adder as indicated. These are called internal carries.

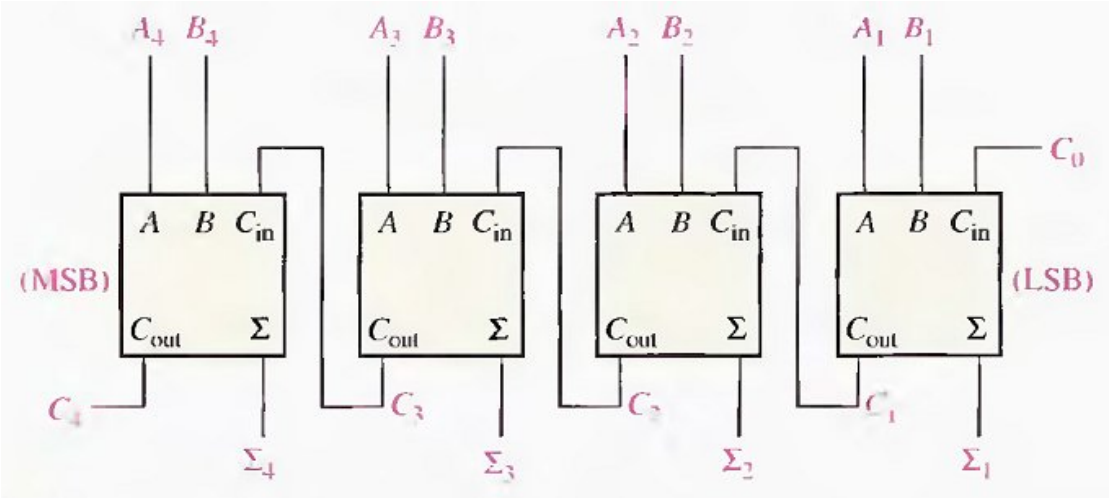


Fig.(7-9)(a)

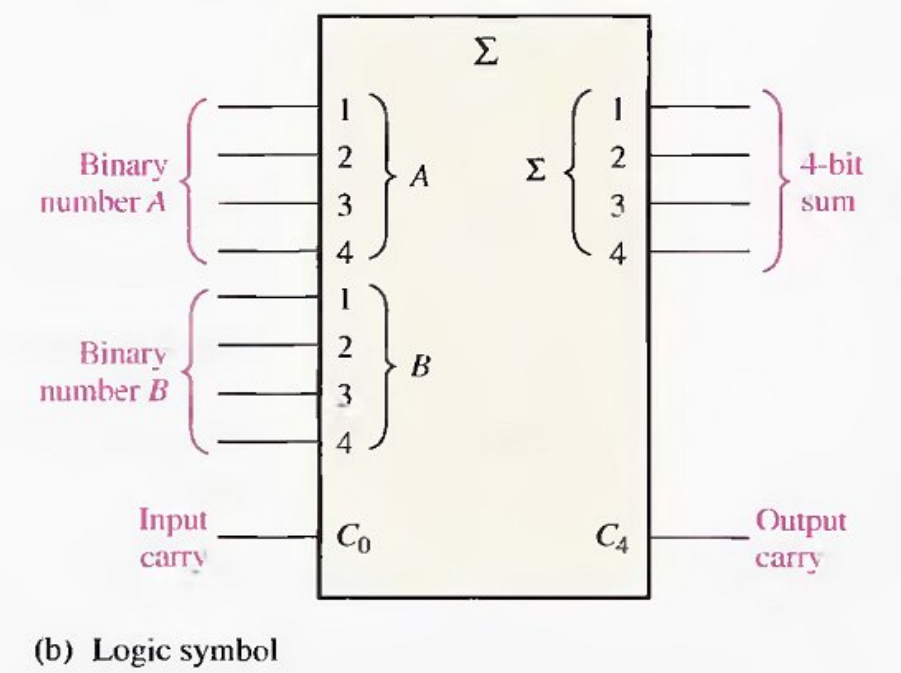


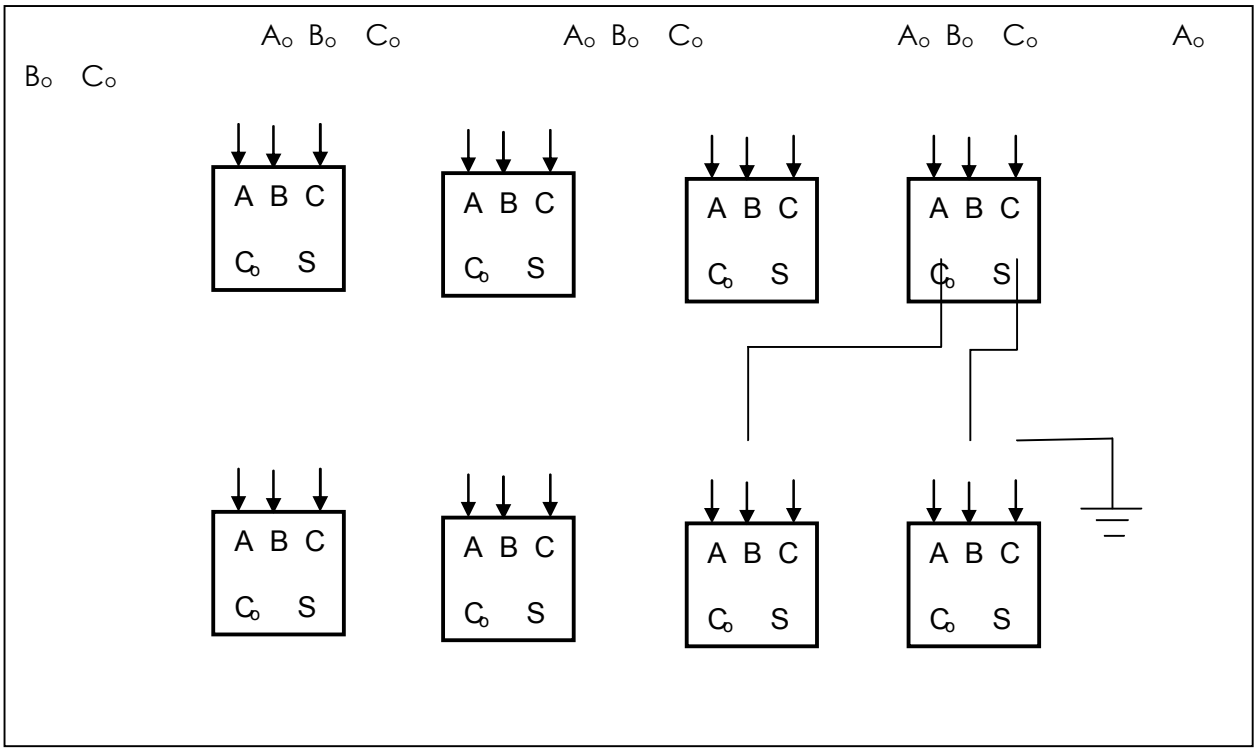
Fig.(7-9) A 4-bit parallel adder.

Carry Save Adder (CSA)

Amethod for adding three or more numbers at a time is called carry-save addition. This process is illustrated in Example below:

Example:

	00011	A
	00001	B
+	**01001**	C
	01011	sum, excluding carries
+	**0001**	carries shifted left one place
	01101	final sum



7-3 COMPARATORS

The basic function of a comparator is to compare the magnitudes of two binary quantities to determine the relationship of those quantities. In its simplest form, a comparator circuit determines whether two numbers are equal.

Equality

The exclusive-OR gate can be used as a basic comparator because its output is a 1 if the two input bits are not equal and a 0 if the input bits are equal. Fig.(7-11) shows the exclusive-OR gate as a 2-bit comparator.

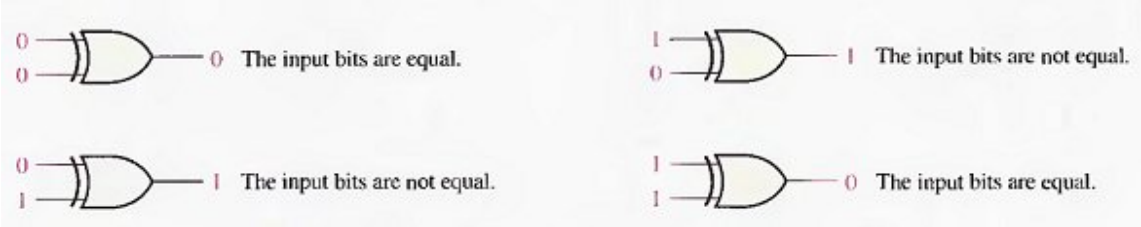


Fig.(7-11) Basic comparator operation.

In order to compare binary numbers containing two bits each, an additional exclusive- OR gate is necessary. The two least significant bits (LSBs) of the two numbers are compared by gate G1. and the two most significant bits (MSBs) are compared by gate G2, as shown in Fig.(7-12). If the two numbers are equal, their corresponding bits are the same, and the output of each exclusive-OR gate is a 0. If the corresponding sets of bits are not equal, a 1 occurs on that exclusive-OR gate output.

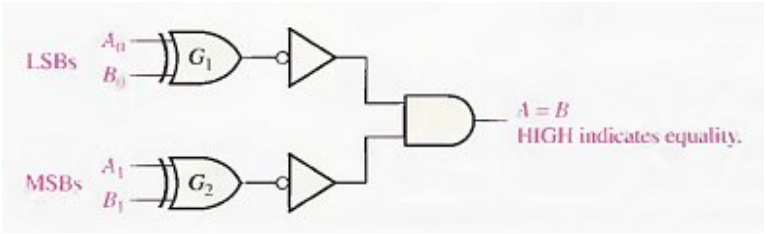


Fig.(7-12) Logic diagram for equality comparison of two 2-bit numbers.

Inequality

In addition to the equality output, many IC comparators provide additional outputs that indicate which of the two binary numbers being compared is the larger. That is, there is an output that indicates when number A is greater than number B (A > B) and an output that indicates when number A is less than number B (A < B), as shown in the logic symbol for a 4-bit comparator in Fig.(7-13).

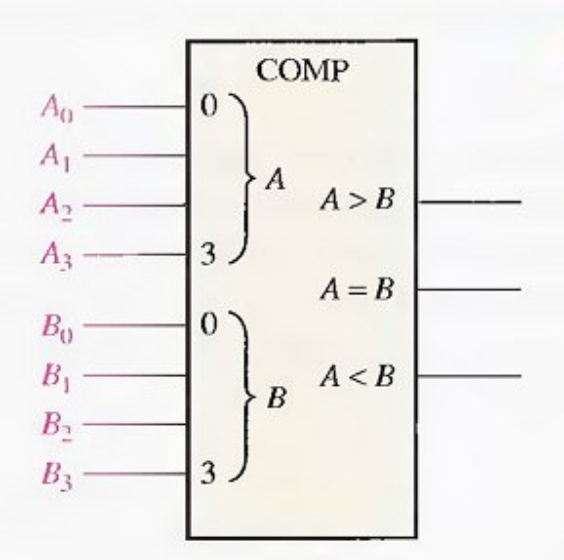


Fig.(7-13) Logic symbol for a 4-bit comparator with inequality indication.

To determine an inequality of binary numbers A and B, you first examine the highest-order bit in each number. The following conditions are possible:

1. If A3 = 1 and B3 = 0, number A is greater than number B.

2. If A3 = 0 and B3 = 1 number A is less than number B.

3. If A3 = B3, then you must examine the next lower bit position for an inequality.